ФИНТЕХ / АРХИТЕКТУРА / БЕЗОПАСНОСТЬ

Надёжный и безопасный финтех

Backend

От HTTP-запроса до подтверждённой финансовой операции

Операции и данные

Транзакции · Повторы · Очереди · Финансовый учёт

Доступ и защита

Подтверждения · Ключи · Секреты · Восстановление

Подготовлено для Алекса · 2 октября 2026 года

Сквозные примеры: заказ карты, денежный перевод и пополнение АТМ

Оглавление

Каждая тема начинается с объяснения механизма. Главы связаны одним процессом: принять намерение, безопасно выполнить действие и установить его результат.

КАК УСТРОЕН ДОКЛАД

Введение. Что меняется, когда мы смотрим со стороны сервера 3

I. КОРРЕКТНАЯ ОПЕРАЦИЯ: ОТ ТРЕБОВАНИЯ ДО ПОВТОРА

1. Ответственность backend: сначала правила, затем компоненты 5
2. Деньги: тип, валюта, точность и округление 7
3. Операция как ресурс: API и жизненный цикл 9
4. Транзакции БД: что действительно происходит при COMMIT 11
5. Конкуренция: одна задача, два сотрудника 13
6. Идемпотентность: как backend узнаёт повтор операции 15
7. Внешний провайдер: таймауты и безопасное продолжение 18

II. МЕЖДУ СИСТЕМАМИ: СООБЩЕНИЯ, ДЕНЬГИ И СВЕРКА

8. Outbox, очереди и saga: работа между системами 20
9. Webhook: как принимать события от партнёра 22
10. Ledger: как учитывать деньги, а не только хранить balance 24
11. Reconciliation: как установить правду после расхождения 27

III. БЕЗОПАСНОСТЬ: ДОСТУП, КЛЮЧИ И ДАННЫЕ

12. Доступ: кто действует и над чьими данными 29
13. OAuth, токены и серверная сессия 31
14. Подтверждение операции: пользователь разрешил именно это действие 33
15. Криптография: что скрывает данные, а что подтверждает их происхождение 35
16. KMS, HSM и секреты: как сервер управляет ключами 37
17. Данные, сеть и опасные входы API 40

IV. ЭКСПЛУАТАЦИЯ И ПОЛНОЕ РЕШЕНИЕ

18. Эксплуатация: скорость, восстановление и наблюдаемость 43
19. Два полных решения: карта и ATM 46
20. Практикум: проверить механизм, а не запомнить название 49

СПРАВОЧНЫЕ РАЗДЕЛЫ

Приложение А. Карта понятий для повторения 52

Приложение Б. Какие стандарты относятся к backend 53

Источники 54

ВВЕДЕНИЕ

Введение. Что меняется, когда мы смотрим со стороны сервера

Пользователь заказывает карту. Сервер банка сохраняет заявку, обращается к партнёру и получает обрыв соединения. Партнёр мог выпустить карту, а мог не получить запрос. Через секунду пользователь нажимает кнопку снова. Ещё через секунду другой экземпляр сервера подхватывает зависшую работу.

Теперь недостаточно правильно нарисовать экран ошибки. Backend должен решить, существует ли уже заказ, кто имеет право увидеть его результат, можно ли повторить внешнее действие и какие записи останутся для расследования. Если заказ платный, добавляется ещё один вопрос: что произошло с деньгами?

Этот доклад — самостоятельная серверная версия предыдущего материала. Из него сохранены сквозные сценарии и порядок объяснения: сначала смысл проблемы, затем устройство механизма, пример и границы его применимости. Главы о базах данных, обработчиках фоновых задач, финансовом учёте и серверных ключах расширены по официальным источникам. Это не описание внутренней архитектуры Revolut и не реконструкция его backend-интервью.

Что здесь является фактом, а что проектным предложением. Правила конкретных продуктов снабжены ссылками на первоисточники. Особенно явно выделены контракты Stripe: правила API v1 и API v2 различаются. Наши endpoint, названия таблиц, состояния, суммы, тайминги и политики — учебные решения. Их нужно адаптировать к реальному продукту. SQL приведён для PostgreSQL; псевдокод объясняет алгоритм и не является готовым платёжным сервисом.

Минимальный словарь участников

Backend — серверная часть продукта: принимает обращения, проверяет права и бизнес-правила, сохраняет данные и взаимодействует с другими системами. **Клиент** — вызывающая её программа: мобильное приложение, сайт или другой сервер. Человека будем называть пользователем. **Провайдер**, или партнёр, — внешняя система, например выпускающая карту.

API, Application Programming Interface, — программный договор о доступных действиях и данных. **HTTP** — протокол запросов и ответов. **Endpoint** — конкретный метод API, например `GET /v1/card-orders/{id}`. **JSON** — текстовый формат объектов, массивов и значений, используемый в примерах. **SDK, Software Development Kit**, — библиотека и инструменты провайдера, упрощающие вызовы API. SDK не знает всех бизнес-правил нашей системы. Общая семантика методов HTTP определяется отдельно от контракта конкретного продукта. [13]

База данных, БД, — система постоянного хранения и обработки данных. **Процесс** — запущенный экземпляр программы. **Экземпляр сервиса** — одна работающая копия backend; таких копий может быть несколько. Переменная в памяти одного процесса не становится общей для остальных и исчезает после его завершения.

Синхронное выполнение означает, что вызывающая сторона ждёт ответ в текущем взаимодействии. **Асинхронное выполнение** означает, что работа может продолжиться после первоначального ответа; результат получают отдельно. Ни один из вариантов сам по себе не гарантирует успех.

Как двигаться по тексту

Главы 1–7 объясняют операции, деньги, API и конкурентный доступ к данным. Главы 8–11 связывают сервер с очередями, партнёрами и финансовым учётом. Главы 12–17 посвящены доступу, подтверждениям, криптографии и защите данных. Главы 18–20 соединяют всё в эксплуатацию, две полные архитектуры и практикум.

Ссылки вида [1] нужны для проверки деталей. Для первого прохода не требуется открывать их все. Полезнее после каждой главы ответить на её заключительный вопрос своими словами. Главный навык — не перечислить технологии, а объяснить, какое правило защищает каждая из них.

I. КОРРЕКТНАЯ ОПЕРАЦИЯ

1. Ответственность backend: сначала правила, затем компоненты

Бизнес-операция и запрос

Бизнес-операция — одно осмысленное действие: выпустить одну карту, перевести 100 GBP определённому получателю, назначить одну задачу сотруднику. **Запрос** — одна попытка передать команду или прочитать результат. Одной бизнес-операции могут соответствовать пять запросов из-за повторов и проверки состояния.

Представим две истории. В первой пользователь один раз подтвердил заказ, но программа трижды отправила запрос. Нужен один заказ. Во второй пользователь через месяц сознательно заказывает дополнительную карту того же дизайна. Нужен новый заказ. Поля «пользователь + дизайн» одинаковы, а смысл различается. Поэтому идентичность операции нельзя автоматически выводить только из похожести данных.

Инвариант — правило, которое должно выполняться во всех допустимых состояниях. В наших примерах: одна задача имеет не больше одного активного исполнителя; одна подтверждённая операция не создаёт лишних заказов; получатель и сумма исполненного перевода совпадают с подтверждёнными; доступный остаток не становится отрицательным, если продукт не допускает перерасход.

Последнее уточнение принципиально: запрет отрицательного остатка — правило конкретного продукта, а не универсальное свойство банковского счёта.

Требования, которые определяют решение

Функциональные требования описывают поведение: создать заказ, узнать статус, отменить ещё не начавшийся выпуск. **Нефункциональные требования** описывают качество: скорость ответа, доступность, допустимую потерю данных. **Scope**, или границы задачи, определяет, какие функции проектируем сейчас.

Для карты сначала выясняем, является ли выпуск асинхронным, доступен ли поиск заказа у партнёра и платит ли пользователь за выпуск. Для АТМ — кто сообщает остаток наличных, кто рассчитывает сумму, как создаётся задача и требуется ли подтверждение завершения банкоматом. От этих ответов зависит больше, чем от выбора названия сервиса.

Модель угроз описывает, что защищаем, от кого и от каких действий. **Граница доверия** — место, где утверждения другой стороны требуют проверки. В нашем продукте телефон не может сам назначить себе привилегии, провайдерский callback не становится достоверным только из-за знакомого поля `orderId`, а внутренний оператор не должен иметь неограниченные права просто потому, что работает в банке. Выбор защиты начинается с такой модели, а не с добавления шифрования без цели. [47]

Почему один backend может быть хорошим началом

Модульный монолит — одно развёртываемое приложение с явно разделёнными модулями. **Микросервис** — отдельно развёртываемый сервис со своей ответственностью. **Развёртывание** — установка и запуск определённой версии программы в рабочей среде.

Для учебного запуска с пятью АТМ выберем модульный backend и одну PostgreSQL. Модули задач, доступа и интеграции с АТМ существуют, но не обязаны общаться по сети. Причина выбора — возможность защищать несколько связанных изменений одной транзакцией БД; устройство транзакции разберём в главе 4.

Это авторское проектное допущение. При независимых командах, резко различающейся нагрузке или отдельной границе безопасности может быть выгодно выделить сервис. Цена такого выделения — дополнительные отказы сети, отдельная эксплуатация и более сложное согласование состояния.

Trade-off, или компромисс, — выигрыш одного свойства ценой другого. Например: отделяем обработчик партнёра от HTTP-обработчика, чтобы запрос пользователя не зависел от долгого ожидания, но теперь обязаны сохранять работу и показывать промежуточное состояние.

> Проверка понимания: можешь ли ты сформулировать три правила своей системы, не используя слова «масштабируемость», «надёжность» и названия технологий?

I. КОРРЕКТНАЯ ОПЕРАЦИЯ

2. Деньги: тип, валюта, точность и округление

Почему сумма — не просто число

Денежное значение включает сумму, валюту и правила точности. `100` может означать сто фунтов, сто пенсов или число, к которому применят иной масштаб. Без контракта это неоднозначные данные.

Minor unit, минимальная единица в денежном API, — единица, в которой сумма кодируется целым числом. Для GBP в наших примерах это пенс. В Stripe валюты без дробных единиц и некоторые операции имеют специальные правила; считать, что любую сумму нужно умножить на сто, нельзя. [7]

Наш контракт:

```
{
  "amountMinor": 125000,
  "currency": "GBP"
}
```

Здесь речь о 1 250 GBP. Клиент и сервер заранее согласуют диапазон целого числа. JSON сам по себе не гарантирует, что каждый язык сохранит любое большое целое без потери точности. Поэтому диапазон и представление являются частью контракта, а не только выбором типа в БД.

Почему не Double

`Double` — число с плавающей точкой в двоичном представлении. Некоторые десятичные дроби, включая 0,1, невозможно представить в нём точно. **Плавающая точка** позволяет хранить широкий диапазон величин, но не обещает точного представления каждой десятичной суммы. Для измерения температуры это часто приемлемо, для точного денежного учёта создаёт ненужную погрешность.

Decimal — десятичное представление с заданной или управляемой точностью. В Java для него используется `BigDecimal`. Создание из строки `"0.10"` сохраняет исходную десятичную величину; создание из уже неточного `double` переносит в объект соответствующее двоичное значение. Деление и изменение масштаба требуют осмысленной политики округления. [15]

```
BigDecimal amount = new BigDecimal("10.05");
BigDecimal rate = new BigDecimal("0.025");
BigDecimal fee = amount.multiply(rate)
    .setScale(2, RoundingMode.HALF_EVEN);
```

В примере **HALF_EVEN** означает округление к ближайшему значению, а ровно посередине — к варианту с чётной последней цифрой. Это иллюстрация, не рекомендация для любой комиссии. Реальный договор может требовать другое правило. Нельзя менять его ради красивого результата теста.

Что хранить в PostgreSQL

PostgreSQL — реляционная БД: данные организованы в таблицы со строками, столбцами и связями. **SQL** — язык обращения к такой БД. `BIGINT` хранит целые числа ограниченного диапазона. `NUMERIC(p,s)` хранит точные десятичные числа: `p` — общая точность, `s` — масштаб, то есть число разрядов после десятичной точки. PostgreSQL различает точные `numeric` и неточные типы с плавающей точкой. [16]

Для проводок нашего GBP-счёта выберем `BIGINT` в пенсах. Для курса конвертации — отдельно подобранный `NUMERIC`, поскольку промежуточному курсу нужна другая точность. Выбор объясняется предметной областью: проведённая сумма и курс — разные величины.

До записи проверяем валюту, знак, допустимую максимальную сумму и отсутствие переполнения. **Переполнение** — выход результата за диапазон выбранного типа. Проверка `amount > 0` не заменяет верхний предел и проверку промежуточного умножения.

Куда девается остаток от округления

Разделим 100 пенсов между тремя получателями. Доли по 33 пенса дают 99, а не 100. Можно распределить 34, 33 и 33 по заранее определённому правилу. Главное — явно обеспечить сохранение общей суммы и воспроизводимость результата.

При конвертации валют появляется **FX quote**, зафиксированное предложение обмена: валюты, курс, комиссии, получаемая сумма и срок действия. Если пользователь подтвердил предложение, сервер не должен незаметно пересчитать его по новому курсу. После истечения срока требуется новое предложение или иной заранее оговорённый процесс.

Нельзя складывать 100 GBP и 100 EUR как обычные числа. Даже при наличии курса перевод между валютами требует отдельного учётного описания. В главе 10 мы увидим, почему балансировать нужно каждую валюту, а не смешанную сумму.

> Проверка понимания: где в твоём решении зафиксированы единицы, диапазон, точность и правило округления? Кто изменяет их при изменении продукта?

I. КОРРЕКТНАЯ ОПЕРАЦИЯ

3. Операция как ресурс: API и жизненный цикл

Зачем сохранять не только результат

Ресурс — объект, который API позволяет адресовать: заказ, перевод, задача. **Идентификатор** отличает один объект от другого. **UUID** — широко используемый формат 128-битного идентификатора; случайная разновидность удобна для создания ID без общего счётчика. Такой ID не является доказательством прав доступа.

Наш заказ имеет собственный `orderId` ещё до успешного выпуска карты. В нём сохраняются выбранный дизайн, снимок адреса и текущее состояние. **Снимок данных** — зафиксированная версия значений на определённый момент. Если пользователь завтра изменит адрес профиля, уже подтверждённый заказ не должен молча изменить адрес доставки.

Машина состояний — модель допустимых состояний и переходов между ними. В учебной модели выпуск проходит `ACCEPTED → ISSUING → ISSUED`; из допустимых промежуточных состояний возможен подтверждённый отказ `REJECTED`. Доставка имеет собственные состояния. Карта может быть выпущена, но ещё не передана курьеру: один статус `SUCCESS` скрывает эту разницу.

Что показывает Stripe PaymentIntent

PaymentIntent у Stripe — сохраняемый объект, описывающий процесс платежа. Он связывает параметры платежа и его изменяющееся состояние, а не сводит платёж к одному HTTP-вызову. Для нашего доклада это пример моделирования жизненного цикла, не рекомендация выбирать именно этот API для любой новой интеграции. [4]

У PaymentIntent есть, например, `requires_payment_method`, `requires_action`, `processing`, `requires_capture`, `succeeded` и `canceled`. Это разные ситуации: нужны новые платёжные данные, действие пользователя, ожидание обработки или отдельное списание ранее авторизованной суммы. Их нельзя корректно передать одним логическим полем `isPaid`. [5]

Авторизация карточного платежа в этом контексте — одобрение платёжной операции с возможным резервированием средств. **Capture** — отдельное подтверждение списания в схеме с раздельными этапами. Это не то же самое, что проверка прав пользователя на API, хотя в обоих случаях встречается слово «авторизация».

Принятие команды и завершение работы

Наше API принимает заказ асинхронно:

```
POST /v1/card-orders
Authorization: Bearer <access-token>
Idempotency-Key: <stable-operation-key>
Content-Type: application/json

{
  "designId": "design_123",
  "deliveryAddressId": "address_456"
}
```

`Authorization` содержит сведения для проверки вызывающей стороны; подробно разберём их в главах 12–13. `Idempotency-Key` помогает связать повторы одной операции; его серверная реализация — в главе 6.

```
HTTP/1.1 202 Accepted
Location: /v1/card-orders/order_741
Content-Type: application/json

{
  "id": "order_741",
  "status": "ACCEPTED",
  "statusUrl": "/v1/card-orders/order_741"
}
```

По HTTP `202 Accepted` означает принятие в обработку, а не её завершение. `201 Created` также может быть уместен, если контракт говорит о синхронном создании ресурса заказа с ещё не завершённым выпуском. Выбор зависит от того, что именно обещает endpoint. [13]

Неопределённость — не бизнес-отказ

Если ответ партнёра потерялся, мы знаем не то, что карта не выпущена, а то, что результат пока не установлен. В нашей модели состояние выпуска остаётся `ISSUING`, а дополнительное поле `recoveryStatus` получает `CHECK_PROVIDER` — требуется проверка у партнёра.

Так отделяются состояние предметного объекта и состояние наших знаний. `MANUAL_REVIEW` будет означать необходимость разбирательства, а не новый вид физической карты. **Терминальное состояние** — состояние, после которого данный процесс больше не меняется обычными переходами. Выбирать его при отсутствии доказательства опасно.

Ошибка как договор с вызывающей стороной

Наш ответ при конфликте назначения:

```
{
  "type": "urn:problem:task-already-assigned",
  "title": "Task is no longer available",
  "status": 409,
  "code": "TASK_ALREADY_ASSIGNED",
  "requestId": "req_821"
}
```

Problem Details — стандартный формат описания ошибок HTTP API; определены поля `type`, `title`, `status`, `detail`, `instance` и возможность расширения. `code` и `requestId` здесь — наши расширения. Формат помогает программе отличать бизнес-конфликт от технического сбоя, но не должен раскрывать секреты и чужие данные. [14]

> Проверка понимания: как клиент узнает результат, если потеряет первоначальный ответ вместе с полученным `orderId`? Одного endpoint чтения по известному ID здесь недостаточно.

I. КОРРЕКТНАЯ ОПЕРАЦИЯ

4. Транзакции БД: что действительно происходит при COMMIT

Транзакция — граница совместного изменения

Транзакция БД — группа действий, которую БД фиксирует как единое изменение или отменяет. `BEGIN` начинает явную транзакцию, `COMMIT` фиксирует её, `ROLLBACK` отменяет незавершённые изменения. Это не синоним финансового перевода: один перевод может проходить через несколько транзакций БД на разных этапах. [17]

Для создания заказа вместе сохраняем сам заказ и сведения, необходимые для продолжения обработки. Если процесс упал до фиксации, не должен остаться «наполовину принятый» заказ. Если фиксация состоялась, другой процесс должен иметь возможность продолжить работу.

ACID — четыре группы свойств транзакций. **Atomicity, атомарность**: изменения фиксируются вместе. **Consistency, целостность**: данные удовлетворяют обеспечиваемым правилам. **Isolation, изоляция**: определяет взаимодействие параллельных транзакций. **Durability, долговечность**: зафиксированные изменения переживают предусмотренные сбои. Буква C не означает, что БД автоматически знает все правила банка: их нужно выразить ограничениями и алгоритмами. [17][18]

Ограничения — правила, которые проверяет сама БД

Constraint, ограничение БД, не позволяет сохранить недопустимые данные. `PRIMARY KEY` определяет основной уникальный идентификатор строки. `UNIQUE` запрещает дубли выбранной комбинации значений. `FOREIGN KEY` проверяет существование связанной записи. `NOT NULL` требует значение. `CHECK` проверяет условие строки. [20]

```
CREATE TABLE refill_tasks (
    id UUID PRIMARY KEY,
    tenant_id UUID NOT NULL,
    status TEXT NOT NULL
        CHECK (status IN ('AVAILABLE', 'ASSIGNED', 'COMPLETED')),
    assignee_id UUID,
    version BIGINT NOT NULL DEFAULT 0,
    CHECK (
        (status = 'AVAILABLE' AND assignee_id IS NULL)
        OR
        (status IN ('ASSIGNED', 'COMPLETED')
            AND assignee_id IS NOT NULL)
    )
);
```

`tenant_id` обозначает организацию или изолированную область данных; правила доступа к ней разберём в главе 12. Это сокращённая учебная схема: внешние ключи к сотрудникам и организациям, история назначения и прочие поля здесь опущены.

Ограничение проверяет согласованность статуса и исполнителя, но само не решает, имеет ли конкретный сотрудник право взять задачу. Оно также не задаёт все допустимые переходы: для них нужен контролируемый путь изменения данных.

Изоляция: какую картину видят параллельные операции

Конкурентное выполнение — пересекающиеся во времени операции. **MVCC, многоверсионное управление конкурентным доступом**, позволяет БД хранить версии строк и показывать транзакциям подходящую картину данных.

В PostgreSQL при **Read Committed** обычное чтение видит зафиксированные данные на начало своего SQL-оператора; следующий оператор может увидеть более новые изменения. При **Repeatable Read** используется стабильный снимок транзакции, но сложные взаимные нарушения бизнес-правил всё ещё возможны. **Serializable** контролирует выполнение так, чтобы успешно зафиксированные транзакции были совместимы с некоторым последовательным порядком; конфликт может закончиться ошибкой и необходимостью повторить транзакцию целиком. [18]

Итак, `BEGIN` вокруг «прочитал свободную задачу — назначил» ещё не является доказательством исключительности. Нужен механизм разрешения конфликта. В следующей главе используем условное обновление.

Почему COMMIT тоже может иметь неизвестный результат

Сервер БД мог зафиксировать транзакцию, а приложение потеряло соединение до получения подтверждения. Отсутствие ответа не равнозначно откату. Здесь снова требуется стабильная идентичность операции и чтение сохранённого результата при восстановлении.

WAL, Write-Ahead Log, — журнал упреждающей записи: БД сначала записывает сведения об изменениях в журнал, а затем может обновить основные страницы данных. Он используется для восстановления. Гарантии зависят от конфигурации: например, отключение `synchronous_commit` допускает потерю недавно подтверждённых транзакций при определённых сбоях. [25][28]

Для финансовых изменений нельзя без обсуждения снижать долговечность ради быстрого ответа. И всё же даже надёжный локальный COMMIT не обещает сохранность при любой региональной катастрофе: копирование и восстановление — отдельная тема.

> Проверка понимания: какие действия реально входят в одну транзакцию твоей БД, а какие только нарисованы рядом? Внешний HTTP-вызов не откатывается командой ROLLBACK.

I. КОРРЕКТНАЯ ОПЕРАЦИЯ

5. Конкуренция: одна задача, два сотрудника

Ошибка «сначала проверим, потом изменим»

Сотрудники А и Б одновременно читают `status = AVAILABLE`. Оба проходят проверку в программе и затем обновляют строку. Последняя запись может перетереть первую. **Race condition**, гонка, — ошибка, при которой результат зависит от непредусмотренного взаимного порядка действий.

Даже если у каждого запроса свой idempotency key, это две разные операции. Защита от повторов не обязана подавлять одну из них. Нам нужно другое свойство: только одна команда может изменить доступную задачу на назначенную.

Условное обновление

В нашем контракте действующий сотрудник определяется из проверенной сессии. Право работать с организацией и задачей проверяется сервером. Затем используем параметризованный SQL: `:name` здесь обозначает значение, подставляемое драйвером, а не строковую склейку.

```
UPDATE refill_tasks
SET status = 'ASSIGNED',
    assignee_id = :actor_id,
    version = version + 1
WHERE id = :task_id
  AND tenant_id = :tenant_id
  AND status = 'AVAILABLE'
RETURNING id, assignee_id, status, version;
```

Проверка состояния и запись выполняются одним оператором. Для конкурентного обновления одной строки PostgreSQL в Read Committed после ожидания проверяет условие на актуальной версии. Поэтому в этом примере одна команда получит строку, а другая не обновит уже занятую задачу. [18]

Ноль возвращённых строк ещё не говорит, какая именно причина произошла. Задача могла не существовать, быть недоступной этому пользователю или уже иметь исполнителя. Дальнейшее чтение выполняем с теми же правилами доступа и формируем безопасный ответ. Нельзя через различия сообщений раскрыть чужую задачу.

Если тем же сотрудником повторяется уже успешное назначение, наш API может вернуть актуальное назначение как успех, а не новый конфликт. Но уведомление и запись истории при этом не должны создаваться снова. Это отдельная семантика повторного вызова, которую нужно определить явно.

Пессимистическая и оптимистическая защита

Пессимистическая блокировка временно не допускает конфликтующие изменения. `SELECT ... FOR UPDATE` блокирует выбранные строки до завершения транзакции. Она полезна, когда нужно прочитать несколько связанных значений, проверить правило и затем изменить их. [19]

Оптимистическая проверка версии предполагает, что конфликт редок: программа читает `version = 7`, а записывает только при всё ещё равной семи версии. Ноль обновлённых строк означает, что данные изменились и решение нужно пересчитать. Поле версии помогает обнаружить устаревшее действие, но не заменяет проверки прав и бизнес-состояния.

Для нашей простой команды достаточно условия `status = AVAILABLE`. Для редактирования сложной заявки версия позволит не затереть чужие изменения. Это авторский выбор по сложности конкретного правила.

Deadlock и повтор транзакции

Deadlock, взаимная блокировка, возникает, когда транзакция А держит строку X и ждёт Y, а транзакция Б держит Y и ждёт X. PostgreSQL обнаруживает такие циклы и прерывает одну из транзакций. Риск уменьшают короткие транзакции и одинаковый порядок захвата объектов. [19]

Для перевода между счетами можно блокировать обе записи в порядке их идентификаторов. После ошибки повторяют целое вычисление с новым чтением, а не только последний SQL-оператор. Внешний выпуск карты нельзя помещать внутрь такой автоматически повторяемой части: он не откатится вместе с БД.

Если назначение хранится отдельной таблицей

Допустим, история назначений сохраняется строками с `ended_at`. **Частичный уникальный индекс** проверяет уникальность только строк, удовлетворяющих условию. Так можно запретить две активные записи на одну задачу:

```
CREATE UNIQUE INDEX one_active_assignment
ON task_assignments (tenant_id, task_id)
WHERE ended_at IS NULL;
```

Индекс — структура БД для поиска и, в данном случае, ограничения уникальности. Исторические назначения с заполненным `ended_at` не мешают созданию нового. PostgreSQL поддерживает такие частичные индексы. [23]

> Проверка понимания: что произойдёт, если одновременно придут два разных запроса с разными ключами? Назови конкретный SQL-оператор или ограничение, которое разрешает конфликт.

I. КОРРЕКТНАЯ ОПЕРАЦИЯ

6. Идемпотентность: как backend узнаёт повтор операции

Что означает это слово

Идемпотентность означает, что повтор одного и того же действия не добавляет эффектов сверх первого выполнения. Присвоить задаче состояние «закрыта» потенциально идемпотентно; каждый раз создавать новую запись заказа — нет. Общая семантика HTTP считает безопасные методы, а также PUT и DELETE идемпотентными, но не обещает этого любому POST. Идемпотентность не требует одинакового текста ответа на каждом повторе. [13]

Idempotency key, ключ идемпотентности, — идентификатор одного намерения, передаваемый для распознавания его повторных попыток. Сам по себе заголовок ничего не защищает: серверу нужны долговременная запись, проверка совпадения параметров и атомарное исключение конкурирующего создания.

Для получения списка отдельный ключ не нужен. Для создания заказа карты он полезен, потому что потерянный ответ естественным образом приводит к повтору. Для назначения существующей задачи нужное поведение иногда можно обеспечить через состояние самой задачи. Отдельный ключ выбирается по семантике, а не только по HTTP-методу.

Конкретный контракт Stripe API v1

Stripe v1 сохраняет код и тело первого результата после начала выполнения endpoint и возвращает сохранённый результат при повторе, включая `500`. Ключ сравнивается с исходными параметрами. Ошибки валидации до начала выполнения и определённые конкурентные конфликты не сохраняются как результат операции. Документация допускает удаление ключей после как минимум 24 часов: повтор после удаления способен начать новое выполнение. Это не бессрочная защита. [1]

Отсюда важный вывод для интеграции: нельзя самостоятельно решить, что новый ключ «исправит» неопределённую операцию. Он может просто снять защиту от повторного эффекта.

Почему нельзя смешивать v1 и v2

У Stripe API v2 другой контракт: повтор успешного запроса не создаёт новых изменений, но может вернуть обновлённый ответ; после ошибки API может повторить неудавшуюся часть. Для распознавания повтора учитываются ключ, API, область account или sandbox и окно 30 дней. **Sandbox** — изолированная тестовая среда. В v2 ключи применимы к POST и DELETE. Эти правила относятся к соответствующему пространству API, а не ко всем методам Stripe одновременно. [2]

Наш учебный backend не обязан копировать ни один контракт буквально. Но он обязан столь же явно документировать повторы, ошибки, область ключа и срок хранения.

Область ключа и совпадение команды

Scope ключа, область его уникальности, отвечает на вопрос, где одинаковая строка означает одну операцию. Для нашего self-service API выберем `(tenant_id, actor_id, action, key)`. `actor_id` — проверенный действующий пользователь, `action` — смысл команды, например `CREATE_CARD_ORDER`.

Рядом сохраняем **fingerprint**, отпечаток нормализованной команды. **Нормализация** приводит семантически одинаковые входные значения к одному представлению. Порядок полей JSON не должен превращать повтор в новую команду. И наоборот: другой адрес или дизайн должен выявляться как изменение параметров.

Отпечаток можно получать криптографической хеш-функцией — алгоритмом, вычисляющим компактное значение из данных. Её устройство и ограничения разберём в главе 15. Здесь важнее правило: фиксируем, какие поля участвуют, как представлены отсутствующие значения и какая версия правил использована. Сам отпечаток не заменяет проверку прав и не обязательно делает чувствительные данные анонимными.

Одна короткая транзакция принятия команды

Предлагаемая запись операции содержит область ключа, отпечаток, `operation_id`, ссылку на заказ и первоначальный ответ. Уникальное ограничение в БД разрешает гонку двух серверов. Оба могут предварительно не увидеть запись, но только один сможет создать её в выбранной области.

```
Проверить личность, права и форму запроса.
Получить нормализованные параметры команды.

BEGIN
    Попытаться вставить запись ключа с UNIQUE-ограничением.
    Если ключ уже существует:
        Прочитать запись отдельным SQL-оператором.
        Проверить права на существующий результат.
        Сравнить отпечаток команды.
        При различии подготовить конфликт параметров.
        При совпадении подготовить сохранённый ответ.
    Если ключ новый:
        Проверить текущие бизнес-ограничения.
        Создать заказ и снимок его параметров.
        Сохранить задание на внешний выпуск.
        Сохранить ответ 202 и ссылку на заказ.
COMMIT

Только теперь отправить ответ клиенту.
```

Outbox — таблица исходящих заданий или событий, сохраняемая вместе с бизнес-изменением; в главе 8 подробно разберём, как её обрабатывать. Здесь она нужна, чтобы после принятия заказа работа не потерялась.

В PostgreSQL для попытки вставки можно использовать `INSERT ... ON CONFLICT DO NOTHING RETURNING ...`. Если вставка не произошла, существующую запись читают следующим оператором. Это существенно для Read Committed: конкурентная транзакция могла завершиться только во время ожидания уникальности, и первоначальный снимок её ещё не показывал. [21]

Нельзя считать любое исключение БД признаком дубля. Ошибка соединения, нехватка ресурсов и нарушение другой проверки требуют отдельной обработки. В этом учебном алгоритме окончательная ошибка проверки нового заказа откатывает короткую транзакцию; сохранение отказов — возможная, но отдельно документируемая политика.

Три точки падения

До COMMIT. Ни ключ, ни заказ, ни outbox-задание не зафиксированы. Повтор начинает принятие заново.

После COMMIT, до ответа. Заказ и ответ сохранены. Повтор находит прежний результат. Клиент получает тот же `orderId`, хотя первоначальный ответ не дошёл.

Во время внешнего выпуска. Локальный заказ уже существует. Его защиту нельзя свести к повтору клиентского HTTP-запроса: нужен отдельный стабильный идентификатор вызова партнёра. Это следующая глава.

Обрати внимание: в нашей схеме принятие операции завершено, а выпуск карты ещё нет. Поэтому повтор POST может вернуть прежний `202`; актуальное состояние читается через GET. Так не смешиваются «результат принятия команды» и «текущее состояние заказа».

Почему Redis и срок жизни не решают всё

Redis — отдельное хранилище, часто используемое для быстрых данных и кеширования. **Кеш** — копия или промежуточное представление, которое может быть удалено или устареть. Если только Redis хранит защиту от дубля, а заказ живёт в другой БД, возникает вопрос совместной фиксации этих двух изменений. Скорость поиска не является ответом на него.

TTL, time to live, — срок, после которого запись считается просроченной или удаляется. TTL подходит для временной защиты при ясно определённом окне. Но удаление записи незавершённой операции не доказывает, что внешний эффект отсутствует. Для нашей модели незавершённые операции не очищаются как обычный кеш.

Длительная защита от повторного заказа может дополнительно опираться на устойчивое бизнес-правило: например, только один заказ для конкретного подтверждённого `checkout_id`, то есть ID оформления. Разные idempotency keys не должны обходить это правило, если продукт действительно его требует.

> Проверка понимания: нарисуй падение между сохранением ключа и созданием заказа. Если после него возникает потерянная операция или дубль, эти записи зафиксированы неправильно.

I. КОРРЕКТНАЯ ОПЕРАЦИЯ

7. Внешний провайдер: таймауты и безопасное продолжение

Таймаут сообщает об ожидании, а не об исходе

Timeout, таймаут, — предел ожидания конкретного взаимодействия. **Deadline** — общий срок, до которого мы готовы продолжать весь текущий вызов. **Retry** — новая попытка того же действия после неудачной попытки или отсутствия ответа.

Когда backend перестал ждать, провайдер не обязан остановить обработку. Поэтому timeout после отправки команды переводит нашу операцию в проверку результата, а не автоматически в отказ. Stripe отдельно рекомендует считать результат `500` неопределённым и не повторять такую операцию с новым ключом: первоначальный вызов мог иметь побочные эффекты. [3]

Побочный эффект здесь — внешнее изменение, например выпуск карты или списание денег. Это не оценка важности эффекта, а отличие от простого чтения.

Стабильное намерение и отдельные попытки

Для каждого заказа создаём `provider_operation_id`. Он сохраняется до первого обращения к партнёру и используется во всех повторах этого внешнего действия. **Attempt**, попытка, — одна конкретная отправка: у неё свой номер, время, полученный код и техническая ошибка.

```
order_id: order_741
provider_operation_id: issue_918
attempt 1: отправлен запрос, ответ не получен
attempt 2: повтор с тем же issue_918, получен provider_card_id
```

Номер попытки меняется, идентичность выпуска — нет. Клиентский ключ и ключ провайдера не обязаны быть одной строкой. Их связывает сохранённая операция, а не случайное совпадение.

Перед повтором проверяем, поддерживает ли провайдер идемпотентность, сколько хранит ключ и можно ли получить заказ по нашей ссылке. **External reference** — переданный нами идентификатор, по которому партнёр может связать объект с нашим заказом. Однако отсутствие результата поиска не всегда доказывает отсутствие объекта: поиск может показывать изменения с задержкой.

Если окно защиты у партнёра истекло, а исход всё ещё неизвестен, не делаем слепой повтор. Нужны другой авторитетный способ поиска или операционная сверка. Клиентский заголовок не создаёт гарантию, которой нет у внешнего API.

Backoff, jitter и предел повторов

Backoff — увеличение паузы между попытками. **Exponential backoff** увеличивает её по степенному правилу: например, до 1, 2, 4, 8 секунд. **Jitter** — случайное изменение паузы, чтобы много процессов не повторяли запрос одновременно.

Учебное правило: случайная задержка от нуля до `min(30 секунд, 2^attempt секунд)`. Предел 30 секунд — наш пример, не значение Stripe. Нужны ещё ограничение числа быстрых попыток, общий срок и переход к длительной проверке. AWS рекомендует ограничивать повторы, добавлять jitter и не умножать retry сразу на нескольких слоях. [32]

Если SDK делает три попытки, наш обработчик — три, а вызывающий сервис — ещё три, одна команда способна породить 27 отправок. Поэтому назначаем один контролирующий слой и учитываем поведение SDK. **Retry budget** — предел допустимой дополнительной нагрузки на повторные попытки.

`429` означает ограничение частоты или ресурсов, но конкретную причину и допустимый повтор нужно читать в контракте. Stripe описывает разные виды ограничений; технический ответ об ограничении не означает бизнес-отказ операции. [9]

Circuit breaker: перестать усиливать аварию

Circuit breaker, автоматический размыкатель, временно прекращает обычные вызовы проблемной зависимости. В состоянии closed запросы проходят; при слишком частых отказах он переходит в open; затем half-open допускает ограниченные пробные обращения. Цель — не расходовать все ресурсы на постоянно неработающий сервис. [33]

Он не выполняет компенсацию и не определяет результат ранее отправленного заказа. В нашей системе открытый circuit breaker откладывает новые обращения, но не помечает все находящиеся в обработке заказы как отклонённые.

Кто продолжает работу после падения процесса

Worker, фоновый обработчик, забирает сохранённые задания и выполняет их независимо от пользовательского запроса. **Lease**, временное право обработки, позволяет отметить, что задание занято до определённого времени. После истечения срока другой worker может попробовать продолжить.

Но первый worker мог не умереть, а просто надолго приостановиться. Истечение lease не уничтожает его. Поэтому временное владение уменьшает конкуренцию, но не заменяет защиту внешнего действия: все workers используют тот же `provider_operation_id` и проверяют актуальное состояние перед изменением результата.

> Проверка понимания: что будет, если старый worker проснётся после того, как новый уже продолжил заказ? Наличие таймера ещё не делает выполнение единственным.

II. МЕЖДУ СИСТЕМАМИ

8. Outbox, очереди и saga: работа между системами

Проблема двух записей

Заказ нужно записать в БД и отправить сообщение обработчику. Если сначала зафиксировать заказ, а потом упасть до отправки, сообщение потеряется. Если сначала отправить сообщение, а потом откатить БД, обработчик увидит несуществующий заказ. **Dual write**, двойная запись, — необходимость согласованно изменить две независимо фиксирующие системы.

Transactional outbox решает эту границу так: в одной транзакции записываем бизнес-объект и исходящее сообщение в таблицу той же БД. Отдельный процесс доставляет сообщение. AWS описывает этот паттерн и отдельно предупреждает о возможных дубликатах доставки. [29]

В нашей минимальной архитектуре outbox может быть просто таблицей заданий на выпуск, которую worker читает напрямую. Отдельная система сообщений нужна не ради самого слова outbox, а при конкретных требованиях к маршрутизации, нагрузке или независимым получателям.

Команда, событие и брокер

Команда просит выполнить действие: `IssueCard`. **Событие** сообщает о факте: `CardOrderAccepted` или `CardIssued`. Нельзя называть событием «карта выпущена» то, что пока является только просьбой её выпустить.

Брокер сообщений — промежуточная система, которая принимает сообщения и передаёт их обработчикам. **Очередь** хранит сообщения до обработки по определённым правилам. **Producer** отправляет сообщения, **consumer** принимает. **Acknowledgement, ack**, — подтверждение приёма или обработки в конкретном протоколе.

Например, в RabbitMQ подтверждение публикации брокером и подтверждение обработки consumer — разные механизмы. Первый не доказывает завершения бизнес-действия у получателя. [30]

Откуда берутся дубликаты

Outbox-процесс отправил сообщение, брокер сохранил его, но процесс упал до отметки `sent`. После перезапуска сообщение будет отправлено повторно. Альтернативный порядок — сначала `sent`, потом отправка — создаёт риск потери. Поэтому выбираем повторную доставку и защищаем получателя.

At-least-once означает возможность повторной доставки при обеспечении доставки в рамках оговорённых условий. **At-most-once** допускает потерю, но не повтор в своей границе. **Exactly-once** нужно всегда уточнять: ровно один раз где и какой эффект? Kafka поддерживает определённые транзакционные сценарии внутри своей экосистемы, но запись в стороннюю БД требует отдельного согласования. Название режима брокера не делает единственным выпуск карты у внешнего партнёра. [31]

Для нашего продукта целевое свойство формулируем проще: дубликаты транспорта допустимы, повторный бизнес-эффект одной операции — нет.

Inbox на принимающей стороне

Inbox — долговременная запись принятых сообщений. Получатель сохраняет `event_id` и изменяет свой бизнес-объект одной транзакцией. Уникальность `event_id` позволяет обнаружить повтор; бизнес-ограничение дополнительно защищает случаи разных событий об одном эффекте.

Если запись inbox находится в Redis, а бизнес-эффект в PostgreSQL, проблема двойной записи возникает снова. Если эффект — внешний HTTP-вызов, локальная транзакция тоже не делает его атомарным: нужен такой же протокол операции, как в главе 7.

Как несколько workers делят таблицу заданий

`SELECT ... FOR UPDATE SKIP LOCKED` позволяет пропустить строки, уже заблокированные другими обработчиками. PostgreSQL прямо отмечает полезность этого режима для таблиц, похожих на очередь, и его непригодность для получения общей согласованной картины данных. [22]

В нашем варианте worker короткой транзакцией выбирает задание, записывает временное владение и фиксирует это. Сетевой вызов происходит вне блокировки. Завершая работу, worker проверяет, что обновляет допустимую версию операции.

Fencing token — возрастающий номер владения, который защищаемый ресурс проверяет, отклоняя действия устаревшего владельца. Он полезен только там, где ресурс участвует в проверке. Если API провайдера игнорирует этот номер, сам по себе столбец `lease_version` не защитит внешний выпуск. Там по-прежнему нужен стабильный ключ или другая поддерживаемая провайдером защита.

Saga: согласованный процесс вместо общего отката

Saga — последовательность локальных изменений в разных системах с явно заданным продолжением и компенсациями. **Оркестратор** — компонент, который хранит состояние процесса и решает следующий шаг. **Компенсация** — новое действие, логически исправляющее прежний эффект, а не возвращающее весь мир назад во времени. AWS подчёркивает дополнительные сложности saga, необходимость идемпотентных участников и отсутствие обычной транзакционной изоляции. [34]

Пример: зарезервировали плату за карту, заказали выпуск, организуем доставку. Если доставка недоступна, компенсацией может быть возврат платы или другой способ доставки. Но уже изготовленную физическую карту нельзя «откатить» как SQL-строку. Компенсация тоже способна завершиться неопределённо и должна иметь свой идентификатор, статус и повторную обработку.

Не называем любую последовательность функций saga. В одном модуле с одной БД обычная короткая транзакция часто понятнее. Saga появляется там, где одной такой границы действительно недостаточно.

> Проверка понимания: если сообщение доставлено дважды, где получатель распознает повтор и в одной ли транзакции это происходит с его изменением данных?

II. МЕЖДУ СИСТЕМАМИ

9. Webhook: как принимать события от партнёра

Что такое webhook

Webhook — HTTP-запрос, который внешняя система отправляет на наш endpoint при событии. Вместо постоянного вопроса «карта уже выпущена?» партнёр сообщает об изменении сам. Это другой способ доставки информации, а не привилегированный канал, которому можно без проверки доверять.

Payload — полезное содержимое сообщения; в нашем случае JSON с идентификатором события и данными объекта. **Подпись webhook** — проверяемое криптографическое значение, связывающее содержимое с отправителем, владеющим соответствующим секретом или ключом. Проверка по официальной библиотеке предпочтительнее самодельного разбора.

Stripe требует проверять подпись с исходным, не изменённым телом запроса, описывает защиту по времени, возможность повторной доставки и отсутствие гарантии порядка событий. Эти свойства нужно учитывать независимо друг от друга. [6]

Сначала принять надёжно, затем обработать

Предлагаемый порядок нашего endpoint:

```
Ограничить размер запроса.
Получить исходные байты тела.
Проверить подпись и допустимость времени.
Проверить ожидаемую среду и аккаунт провайдера.
Сохранить событие в inbox с уникальным ID.
Зафиксировать транзакцию.
Ответить успешным HTTP-кодом.

Отдельный worker применяет событие к заказу.
```

Если БД недоступна, событие ещё не принято надёжно. Мы не отвечаем успехом только потому, что получили байты в оперативную память. Повтор уже сохранённого события, напротив, можно быстро подтвердить, не выполняя эффект заново.

Replay attack, повторная атака, — повторное использование перехваченного корректного сообщения. Проверка свежести уменьшает допустимое окно атаки, а уникальность ID исключает повторную обработку уже принятого события. Ни одна из этих проверок не подменяет другую.

Порядок событий и актуальное состояние

Получили `issued`, а потом запоздалое `issuing`. Без проверки переходов последнее сообщение откатит заказ в прошлое. В нашем решении допускаются только согласованные переходы, а при противоречии запрашивается авторитетный объект у провайдера.

Версия объекта — число или другой маркер, который позволяет сравнить последовательность изменений, если провайдер действительно обещает такой смысл. Просто время доставки или случайный `event_id` версией не являются. Нельзя отсортировать случайные UUID и объявить полученный порядок порядком событий.

У провайдера могут быть разные события об одном факте. Поэтому `UNIQUE(event_id)` защищает транспортный дубль, а ограничение вроде «для этого заказа карта уже связана» защищает бизнес-эффект. Это разные уровни дедупликации.

Не хранить лишнее и не обрабатывать всё подряд

Дедупликация — обнаружение повторных представлений одного сообщения или действия. Для inbox сохраняем ID, тип события, область провайдера, нужные данные и состояние обработки. Полное исходное тело иногда необходимо для расследования, но оно может содержать чувствительные данные; тогда нужны защита и определённый срок хранения.

Неизвестный тип события не должен произвольно менять заказ. Изменения контракта проверяются на тестовых данных. Если событие ссылается на незнакомую карту, ищем связь с нашей сохранённой операцией, а не создаём право доступа по присланному `userId`.

Webhook ускоряет получение результата, но не является единственным механизмом восстановления. Периодическая проверка незавершённых операций нужна, чтобы отсутствие одного уведомления не оставляло заказ неизвестным навсегда.

> Проверка понимания: что произойдёт, если обработчик упадёт после успешного HTTP-ответа партнёру? Покажи запись, которая позволит продолжить обработку.

II. МЕЖДУ СИСТЕМАМИ

10. Ledger: как учитывать деньги, а не только хранить balance

Что такое ledger

Ledger, финансовая книга, — система учётных записей, объясняющих движение и принадлежность денег. **Ledger account**, учётный счёт, — объект, по которому ведётся определённый остаток. Это не обязательно банковский счёт с реквизитами: это может быть обязательство перед пользователем, деньги у партнёра или доход от комиссии.

Проводка, ledger entry, — одна сторона учётного изменения на конкретном счёте. **Ledger transaction**, учётная транзакция, объединяет связанные проводки. **Двойная запись** требует согласованной записи обеих сторон: сумма дебетов равна сумме кредитов в соответствующей валюте и учётной модели. Modern Treasury использует атомарную группу проводок и неизменяемость финансовых полей проведённой транзакции. [35]

Один столбец `balance` отвечает на вопрос «сколько сейчас», но сам по себе не объясняет «почему столько». В учебной системе остаток связан с журналом проводок; при необходимости храним быстро читаемое значение, однако обновляем его согласованно и умеем сверить с историей.

Дебет и кредит без ложного правила «плюс и минус»

Дебет и **кредит** — стороны учётной записи. Их эффект зависит от типа счёта. **Актив** — например, деньги, которыми располагает организация. **Обязательство** — сумма, которую организация должна другому лицу. Дебет обычно увеличивает актив и уменьшает обязательство; кредит — наоборот. Поэтому «дебет всегда минус» является ошибкой. [36]

Рассмотрим упрощённую модель кошелька. Компания держит пользовательские средства, а учётные счета клиентов отражают её обязательства перед ними. Пользователь А переводит 100 GBP пользователю Б внутри этой системы:

Учётный счёт	Дебет	Кредит	Смысл
Обязательство перед А	100 GBP	—	Должны А на 100 меньше
Обязательство перед Б	—	100 GBP	Должны Б на 100 больше

Деньги компании на внешнем банковском счёте от такого внутреннего перевода не переместились. Изменилось распределение обязательств. Это учебная модель, а не универсальный план счетов банка.

Если компания получила 100 GBP на банковский счёт для пополнения кошелька А, другая запись увеличивает актив «средства в банке» по дебету и обязательство перед А по кредиту. Именно контекст счетов объясняет движение, а не знак одного поля.

Баланс и резерв

Posted, проведённая запись, — зафиксированное финансовое изменение в выбранной модели. **Pending** — ещё не завершённое изменение. **Available balance**, доступный остаток, — сумма, которую продукт разрешает использовать с учётом действующих правил. Modern Treasury различает соответствующие балансы и состояния; нельзя считать их взаимозаменяемыми. [37]

Hold, резерв, временно исключает сумму из повторного расходования. В нашей упрощённой модели без кредитного лимита:

```
доступный остаток = проведённый остаток - активные резервы
1 000 GBP - 150 GBP = 850 GBP
```

Это не универсальная формула любого банка: поступления в обработке, кредитные лимиты и другие ограничения могут менять расчёт.

Для карточных платежей Stripe поддерживает отдельные авторизацию и capture там, где это допускается конкретным способом платежа. Резерв имеет ограниченный жизненный цикл; сроки и возможности нужно брать из фактического контракта, а не зашивать «всегда семь дней». [69]

Два одновременных списания

Пусть доступно 100 GBP. Два перевода по 80 GBP имеют разные идентификаторы и оба законны по отдельности. Ключ идемпотентности каждого не решает суммарный перерасход.

В нашем предложении проверка доступности и создание резерва выполняются одной транзакцией с контролем конкурентного доступа к соответствующему счёту. После первого резерва второй запрос видит только 20 GBP или проигрывает условное обновление. Результат проверки нельзя брать из запаздывающей копии баланса.

Пример самого принципа:

```
UPDATE account_limits
SET reserved_minor = reserved_minor + :amount
WHERE account_id = :account_id
  AND currency = :currency
  AND posted_minor - reserved_minor >= :amount
RETURNING reserved_minor;
```

Это лишь часть алгоритма. В той же транзакции должны появиться уникальный резерв для операции и необходимые учётные записи. `:amount` предварительно проверяется как положительное значение допустимого диапазона. Любое альтернативное изменение остатка обязано следовать той же модели блокировок и ограничений.

Балансировка проводок сама по себе не запрещает перерасход. И наоборот, проверка неотрицательного остатка не доказывает правильность всех проводок. Это два разных инварианта.

Как исправлять уже проведённое

Reversal, обратная учётная операция, компенсирует эффект предыдущей новыми проводками со ссылкой на оригинал. Она не стирает историю. В модели Modern Treasury проведённые финансовые поля не редактируются, а обратный эффект оформляется новой транзакцией; возможность изменить metadata не равнозначна праву переписать суммы. [37]

Metadata — вспомогательные поля: например, внутренний идентификатор заказа или пояснение. Они не должны превращаться в скрытый источник финансовой истины.

Refund, возврат платежа, — самостоятельная операция возврата всей суммы или части. Stripe допускает несколько частичных возвратов, но их сумма не должна превышать исходный платёж; возврат может иметь собственные промежуточные и неуспешные состояния. [67]

В нашем API каждый возврат получает свой ID. Одновременные возвраты должны проверять общий доступный к возврату остаток с учётом уже зарезервированных попыток. Timeout возврата не означает, что его сумму можно тут же вернуть другим запросом.

Dispute, спор по платежу, — отдельный процесс оспаривания через платёжную систему, а не обычная команда возврата продавца. У Stripe он имеет собственный жизненный цикл. Поэтому «платёж когда-то succeeded» не означает отсутствия последующих финансовых событий; они записываются отдельно. [68]

Валюты и ограничения схемы

Проводки 100 GBP и 120 EUR нельзя объявить равными только потому, что использован курс 1,2. В нашей модели каждая валюта балансируется отдельно через соответствующие обменные счета; курс, комиссия и округление сохраняются как часть операции.

Обычный построчный `CHECK` не проверяет произвольную сумму проводок по другим строкам. Нужен контролируемый способ проведения: процедура или сервис с транзакцией, блокировками, проверкой всей группы и запретом обходных записей. PostgreSQL отдельно предупреждает об ограничениях межстрочных проверок в CHECK. [20]

> Проверка понимания: можешь ли ты восстановить причину каждого изменения остатка? И можешь ли доказать, что два разных перевода не потратят одни и те же доступные деньги?

II. МЕЖДУ СИСТЕМАМИ

11. Reconciliation: как установить правду после расхождения

Что такое сверка

Reconciliation, сверка, — сопоставление независимо полученных данных, чтобы обнаружить и объяснить расхождения. Это не просто повтор HTTP-запроса. Modern Treasury описывает сверку внутреннего учётного счёта с балансом банка или провайдера и поиск неотражённых движений. [38]

Наша БД хранит намерение перевести 100 GBP. Партнёр хранит факт исполнения. Выписка отражает реальное движение в своём периоде. В идеале сведения связаны, но могут прибывать в разное время и с разной детализацией. Сверка проверяет эти связи.

Различаем **операционную сверку** — выяснить исход конкретного заказа или платежа — и **финансовую сверку** — сопоставить учтённые суммы и движения с внешними данными. Заказ карты без оплаты тоже требует первого вида.

Почему итоговые суммы недостаточны

Во внутреннем учёте пропущено списание 100 GBP и ошибочно продублировано другое списание 100 GBP. Общая сумма совпадает с выпиской. Но два клиента видят неправильные операции. Поэтому сравниваем не только итоги, но и идентификаторы, валюты, направление, сумму, область провайдера и связь с исходным намерением.

Settlement, расчёт по платежам, — этап реального урегулирования денежных обязательств между участниками. В учебном примере отчёт провайдера может объединять несколько платежей и вычитать комиссии. Нельзя сравнить чистую сумму перечисления с суммой всех заказов без понимания этих составляющих.

Cut-off, граница отчётного периода, задаёт, какие движения попадают в конкретный отчёт. Время события, время его получения нами и учётная дата — разные значения. Если смешать их, нормальная задержка выглядит как пропажа денег.

Учебный процесс сверки

Сначала сохраняем внешний отчёт с идентификатором источника и периода. Затем нормализуем записи, сохраняя оригинальные значения для проверяемости. По устойчивым связям находим соответствующие операции. Результат каждой строки — совпадение, ожидаемая задержка, несовпадение суммы или отсутствие пары.

Reconciliation case, случай расхождения, — отдельная запись для расследования с владельцем, причиной, доказательствами и статусом. Мы не исправляем баланс произвольным UPDATE только ради нуля в отчёте. Если нужна корректировка, оформляем её через контролируемую финансовую операцию с аудитом.

```
Внешняя запись: provider_payment_52, 100 GBP, выполнено
Внутренняя запись: transfer_741, 100 GBP, результат неизвестен
Связь: provider_operation_id = payout_918
Действие: подтвердить исход прежнего перевода
Не действие: создать новый перевод на 100 GBP
```

Повторная загрузка того же отчёта тоже должна быть безопасной. Сохраняем идентификаторы уже обработанных источников и строк. Пустой ответ из внешнего списка не доказывает отсутствия операций, пока не проверены доступ, страница, период и успешность получения.

Что делать с долго неизвестной операцией

Escalation, эскалация, — передача случая на следующий уровень решения по установленному правилу. В нашей модели после ограниченного автоматического поиска операция остаётся видимой в очереди расследований. У неё есть срок следующей проверки и ответственный, а не просто забытый `UNKNOWN`.

Если провайдер не поддерживает ни безопасный повтор, ни надёжный поиск, backend не может честно объявить одновременно «точно не будет дублей» и «точно завершим любое действие автоматически». В такой ситуации мы выбираем ограниченное автоматическое выполнение и ручную сверку неопределённых случаев. Это ограничение конкретного интерфейса, которое должно быть известно продукту.

Обратная сторона безопасности — зависшие резервы. Снимать их просто по техническому таймауту нельзя, если внешнее списание могло состояться. Но бессрочно удерживать деньги без процесса разбора тоже плохое решение. Поэтому резерв, проверка исхода и эскалация проектируются вместе.

> Проверка понимания: какое доказательство позволяет твоей системе перевести неизвестную операцию в успех или отказ? Истечение времени само по себе таким доказательством не является.

III. БЕЗОПАСНОСТЬ

12. Доступ: кто действует и над чьими данными

Аутентификация и авторизация

Аутентификация проверяет личность вызывающей стороны. **Авторизация доступа** проверяет, разрешено ли этой стороне конкретное действие над конкретным объектом. Успешный вход не даёт права читать все заказы или списывать с любого счёта. OWASP рекомендует минимальные права, запрет по умолчанию и проверку разрешений при каждом запросе. [39]

Actor, действующий субъект, — пользователь или сервис, выполняющий действие. **Principal** — его проверенное представление в программе: ID, область, права и другие подтверждённые атрибуты. Клиентский `userId` в теле не превращается в principal.

Для «назначить задачу себе» получаем исполнителя из проверенного контекста. Для «назначить другого сотрудника», доступного руководителю, `assigneeId` в теле нужен, но отдельно проверяются полномочия руководителя и допустимость указанного исполнителя. Запрет относится к доверию неподтверждённому утверждению, а не к буквам `userId` в JSON.

Tenant: граница данных организации

Tenant — логически изолированная организация или другая область клиентов в общей системе. **Multi-tenancy** означает обслуживание нескольких таких областей одним продуктом. В нашем АТМ-примере сотрудник организации А не должен видеть задачи организации Б.

Проверку области проводим при чтении, изменении, экспорте и восстановлении операции. Кеш результата и область ключей идемпотентности также учитывают tenant. Иначе правильная проверка основного endpoint может быть обойдена через вспомогательный путь.

BOLA, Broken Object Level Authorization, — уязвимость, при которой API не проверяет доступ к запрошенному объекту. Она возможна и с UUID: непредсказуемость ID не заменяет авторизацию. OWASP отмечает необходимость проверки конкретного действия над конкретным объектом. [40]

Роль, атрибуты и отношения

RBAC, Role-Based Access Control, разрешает действия на основе роли: например, «сотрудник», «руководитель», «оператор». **ABAC, Attribute-Based Access Control**, учитывает атрибуты субъекта, объекта и ситуации. **ReBAC, Relationship-Based Access Control**, использует отношения: например, принадлежит ли счёт пользователю или входит ли сотрудник в обслуживающую команду. Это модели выражения политики, а не взаимоисключающие продукты. [39]

Для завершения АТМ-задачи роли «сотрудник» недостаточно. В нашем варианте нужны одновременно нужная организация, назначение именно этому сотруднику и допустимое состояние задачи. На уровне кода это явная проверка, а не скрытая надежда на экран приложения.

Защита внутри БД

RLS, Row-Level Security, — правила БД, ограничивающие доступ к строкам. PostgreSQL позволяет применять такие политики, но некоторые привилегированные роли и обычно владелец таблицы обходят их; конфигурация роли приложения важна. [24]

RLS может быть дополнительным барьером против случайно забытого tenant-фильтра, но не избавляет от модели доступа. Если приложение само передаёт в БД произвольный tenant без проверки, или подключено ролью с обходом RLS, наличие политики на диаграмме мало что значит.

Администраторы и разделение обязанностей

Least privilege, минимальные привилегии, означает выдавать ровно те полномочия, которые нужны работе. **Separation of duties**, разделение обязанностей, не позволяет одному субъекту бесконтрольно выполнить все чувствительные этапы.

Учебный пример: оператор может предложить корректировку учёта, другой уполномоченный сотрудник подтверждает её, а исполнение создаёт аудируемую операцию. Такая схема нужна не для любого действия, а там, где риск оправдывает задержку и эксплуатационную сложность.

Audit trail, след аудита, — последовательность записей о значимых действиях: кто, над чем, с каким основанием и результатом. Это не просто текст технического исключения. Его сохранность и доступ рассмотрим в главе 17.

> Проверка понимания: проверяется ли доступ при повторе уже успешного запроса? Сохранённый ответ не должен стать способом прочитать объект после утраты прав.

III. БЕЗОПАСНОСТЬ

13. OAuth, токены и серверная сессия

Что такое сессия и токен

Сессия — ограниченный во времени контекст взаимодействия после входа: кто вошёл, какие проверки прошёл и действителен ли ещё доступ. **Токен** — значение, предъявляемое для определённой проверки или полномочия. Это общее слово, а не единый формат: токен заказа карты, access token и одноразовое подтверждение имеют разные назначения.

Access token даёт возможность обратиться к определённым ресурсам. **Bearer token** можно использовать, предъявив само значение; поэтому его утечка опасна. **Refresh token** позволяет получить новый access token по правилам сервера авторизации. У него другая аудитория и другая функция: API заказов не должно принимать refresh token вместо access token. [41]

Что решает OAuth

OAuth 2.0 — протокол предоставления ограниченного доступа, в котором **authorization server**, сервер авторизации, выдаёт токены, а **resource server**, сервер ресурсов, защищает API. Пользователь, клиентская программа и сервер ресурсов — разные роли. OAuth сам по себе не является универсальным протоколом удостоверения личности. [41]

OpenID Connect, OIDC, добавляет к OAuth протокол входа и **ID token** — подтверждение сведений об аутентификации для клиента. ID token не следует автоматически принимать как пропуск в произвольное API: его назначение отличается от access token. [45]

В нашем учебном продукте не реализуем собственный OAuth-сервер ради упражнения. Берём поддерживаемую реализацию и проектируем границы доверия, проверки и работу с сессией. Замена реализации не отменяет необходимости понимать, что именно проверяется.

JWT: подписанные данные, а не обязательно секретные данные

JWT, JSON Web Token, — формат утверждений о субъекте и условиях использования. **Claim** — отдельное утверждение, например идентификатор субъекта или срок действия. В распространённом подписанном варианте токен содержит заголовок, данные и криптографическую подпись.

Цифровая подпись позволяет проверить связь данных с владельцем ключа и обнаружить изменение: отправитель вычисляет её по сообщению закрытым ключом, а получатель проверяет соответствующим открытым. Подробнее это различие разберём в главе 15. Подписанный JWT обычно читается получателем без расшифрования: **Base64url**, используемое текстовое кодирование байтов, не является шифрованием. Секретные поля не следует помещать туда только потому, что строка выглядит непонятно. [75]

Сервер использует доверенную библиотеку и заранее настроенную политику: допустимые алгоритмы, ключи, ожидаемые `issuer` и `audience`, срок действия и требуемые утверждения. **Issuer** — кто выдал токен, **audience** — кому он предназначен. RFC 8725 подчёркивает проверки алгоритмов, эмитента и аудитории; простое декодирование не является проверкой. [43]

`sub` обозначает субъект, `exp` — время истечения, `nbf` — время, ранее которого токен не следует принимать. Даже корректная подпись не означает, что роль пользователя не изменилась секунду назад. Это уже вопрос актуальности политики доступа.

Непрозрачный токен и introspection

Opaque token, непрозрачный токен, не раскрывает API структуру утверждений: сервер ищет соответствующее состояние или обращается к издателю. **Introspection** — стандартизованный запрос к серверу авторизации о действительности токена и связанных данных. [44]

JWT позволяет многие проверки выполнять без обращения к издателю, но усложняет немедленный отзыв уже выданного доступа. Непрозрачная сессия удобнее для централизованной проверки отзыва, но добавляет зависимость от хранилища или introspection. Для нашей системы выбор зависит от требований к отзыву и доступности, а не от моды.

Revocation, отзыв, — прекращение действия ранее выданного полномочия. Удалить токен на устройстве недостаточно, если его копия осталась у злоумышленника. Проверяем, какой механизм узнаёт об отзыве: срок действия, состояние сессии, список запрещённых токенов или другая политика. OWASP отдельно рассматривает жизненный цикл и прекращение серверной сессии. [72]

Rotation refresh token

Rotation, ротация, в этом контексте означает выдачу нового refresh token при использовании старого и прекращение обычной действительности прежнего. Это позволяет обнаруживать повторное использование. RFC 9700 для публичных клиентов предусматривает обнаружение replay через rotation или привязку токена к отправителю. **Публичный клиент** не может надёжно скрывать общий секрет внутри распространяемой программы. [42]

Для backend важен одновременный refresh. Два запроса с одним прежним токеном не должны бесконтрольно создавать две независимые ветки сессии. В учебной схеме сервер атомарно проверяет и обновляет состояние семейства токенов. **Семейство токенов** — связанные последовательно выданные токены одной сессии.

Если ответ с новым токеном потерян, нельзя придумывать произвольное «окно прощения» без модели угроз: оно может ослабить обнаружение кражи. Используем документированное поведение выбранного сервера авторизации и проверяем его вместе с клиентом.

Для чувствительных действий долговременная запись операции связывается с actor и основанием разрешения. Фоновая обработка не должна зависеть от хранения сырого bearer token в очереди. Истёкшая пользовательская сессия и уже надёжно принятая команда — разные сущности.

> Проверка понимания: какие именно утверждения ты проверишь после проверки подписи JWT? И что произойдёт с уже выданным доступом после блокировки пользователя?

III. БЕЗОПАСНОСТЬ

14. Подтверждение операции: пользователь разрешил именно это действие

Почему входа недостаточно

Вход доказывает, что пользователь успешно прошёл аутентификацию. Проверка прав доказывает, что ему разрешено пользоваться счётом. **Transaction authorization**, подтверждение конкретной операции, связывает дополнительное разрешение с определёнными параметрами действия. OWASP отдельно требует серверной проверки и защиты параметров от изменения. [46]

Допустим, пользователь видит перевод 100 GBP получателю А. Злоумышленник меняет запрос на 1 000 GBP получателю Б. Если сервер принимает `confirmed: true`, он проверяет только утверждение клиента, а не то, что фактически подтверждалось.

Step-up authentication — дополнительная проверка личности или присутствия пользователя для действия повышенного риска. Она помогает получить более сильное подтверждение, но сама по себе не связывает его с суммой и получателем. Эту связь должен обеспечивать протокол операции.

Challenge и неизменяемый снимок

Challenge, проверочное задание, — сформированные сервером данные, на которые пользователь или его средство подтверждения должны корректно ответить. **Nonce** — значение, предназначенное для однократного использования в определённой криптографической операции или протоколе. В нашем примере это случайная часть challenge.

Предлагаемая серверная последовательность:

```
Подготовить перевод: источник, получатель, валюта,
сумма, комиссия, версия предложения.
Сохранить неизменяемый снимок как transfer_741.
Создать challenge, связанный с этим снимком и actor.
Показать пользователю подтверждаемые значения.
Проверить полученное доказательство на сервере.
Атомарно отметить разрешение использованным
и принять к выполнению именно transfer_741.
```

Proof, доказательство подтверждения, может быть результатом принятого механизма: например, проверенного ответа одноразового средства или подписи зарегистрированным ключом. Этот доклад не задаёт собственную криптографическую схему подписи перевода. Для реальной реализации необходим полный протокол регистрации, отзыва и проверки ключа.

Биометрическое окно на телефоне не является серверным доказательством само по себе. Оно может участвовать в разрешении использования ключа, но backend проверяет результат предусмотренного протокола, а не локальный callback клиента.

Гонка между проверкой и использованием

TOCTOU, time-of-check to time-of-use, — разрыв между моментом проверки и моментом использования проверенных данных. Например, подтвердили версию перевода 7, затем кто-то изменил получателя, а worker исполнил версию 8.

В нашем проекте worker получает неизменяемые параметры принятой операции. В транзакции потребления разрешения проверяются actor, ID операции, версия или отпечаток параметров, срок действия и отсутствие предыдущего использования. Изменение существенных параметров требует нового подтверждения.

Повтор клиентского запроса после потери ответа должен найти уже принятую операцию, а не потребовать снова использовать одноразовое разрешение. Поэтому состояние разрешения и принятие команды фиксируются согласованно. «Одноразовый» не означает «невозможно восстановить успешный результат».

Не путать разрешение и идемпотентность

Идемпотентность отвечает: «это повтор какого действия?». Подтверждение отвечает: «было ли разрешено именно это действие?». Проверка остатка отвечает: «допустимо ли выполнить его сейчас по финансовым правилам?». Ни один из трёх ответов не заменяет остальные.

Подтверждение не обязывает выполнить действие вопреки новым ограничениям. Например, перед отправкой банк может обнаружить блокировку счёта и корректно отказать. Но уже отправленная внешняя операция требует выяснения результата, а не простого удаления разрешения.

> Проверка понимания: какая связь в сохранённых данных не позволяет использовать подтверждение перевода 100 GBP для перевода 1 000 GBP?

III. БЕЗОПАСНОСТЬ

15. Криптография: что скрывает данные, а что подтверждает их происхождение

Шифрование и ключ

Шифрование преобразует открытые данные в зашифрованные так, чтобы восстановление требовало ключа. **Открытый текст, plaintext**, — исходные байты. **Шифротекст, ciphertext**, — результат шифрования. **Криптографический ключ** — секретное либо специальным образом связанное с другой частью пары значение, определяющее работу алгоритма.

Симметричное шифрование использует один секретный ключ для шифрования и расшифрования. Например, наш сервис защищает адрес доставки, а при отправке партнёру расшифровывает его тем же ключом. Защищать нужно и зашифрованный адрес, и доступ к ключу.

Асимметричная криптография использует связанную пару ключей. **Закрытый ключ** сохраняют в секрете; **открытый ключ** предназначен для распространения. Конкретный алгоритм определяет допустимые операции. Пара для подписи позволяет подписывать закрытым ключом и проверять открытым; это не означает, что любой алгоритм подписи одновременно подходит для шифрования. OWASP рассматривает выбор механизмов и управление ключами как отдельные задачи. [47][48]

Что такое цифровая подпись и MAC

Цифровая подпись — значение, вычисленное по сообщению с помощью закрытого ключа. Проверяющий использует соответствующий открытый ключ, чтобы проверить подпись и целостность сообщения. **Целостность** означает отсутствие незамеченного изменения. Подпись не прячет само содержимое и не доказывает автоматически правдивость каждого утверждения подписанта.

MAC, Message Authentication Code, — код проверки сообщения с общим секретом. **HMAC** — его распространённая конструкция на основе хеш-функции. Отправитель вычисляет код, получатель повторяет вычисление и безопасно сравнивает результаты. В отличие от асимметричной подписи, обе стороны с общим секретом способны сформировать корректный код. Поэтому HMAC не даёт того же разделения возможностей, что закрытый и открытый ключи. [74]

Для webhook название «signature» в документации не обязательно означает асимметричную подпись. Важно использовать конкретную схему провайдера, включая байты сообщения и проверку времени, а не выбирать алгоритм по названию заголовка.

Хеширование не является шифрованием

Хеш-функция вычисляет отпечаток фиксированного размера из данных произвольной длины. Криптографическая хеш-функция специально затрудняет поиск исходного значения или другого сообщения с тем же отпечатком. Но короткий известный набор вариантов можно перебрать: хеш шестизначного кода не делает его неприступным.

Пароль обычно нужно проверять, а не восстанавливать. Поэтому его не хранят в обратимо зашифрованном виде ради последующей отправки пользователю. Применяют специальные функции хеширования паролей. **Salt**, соль, — отдельное случайное значение для записи, мешающее эффективно применять заранее вычисленные таблицы. **Argon2id** — специализированная функция, повышающая стоимость перебора через затраты памяти и вычислений; OWASP рекомендует её в современных системах хранения паролей. [49]

Это отличается от API-ключа партнёра: его сервису иногда нужно предъявить в исходном виде, поэтому нужен защищённый механизм хранения секрета. Один и тот же рецепт «всё хешировать» не подходит обоим случаям.

Шифрование с проверкой целостности

AEAD, Authenticated Encryption with Associated Data, — шифрование, одновременно защищающее скрываемые данные и проверяющее их целостность вместе с дополнительным контекстом. **AES-GCM** — распространённая реализация такого режима. **Authentication tag**, тег проверки, позволяет обнаружить изменение шифротекста или связанного контекста. [73]

AAD, additional authenticated data, — дополнительные данные, которые не скрываются, но участвуют в проверке. Например, шифротекст адреса можно связать с `tenant_id`, `order_id` и версией формата. Тогда простая подмена зашифрованного адреса строкой другого заказа не пройдёт проверку, если контекст формируется и проверяется корректно.

Nonce, уже встречавшийся в протоколах подтверждения, здесь — параметр конкретного шифрования. Для AES-GCM недопустимо повторять nonce с тем же ключом. Nonce обычно хранится рядом с шифротекстом и не является секретом. Его требования зависят от алгоритма: «одинаковые данные — значит одинаковый nonce» является опасным правилом. [73]

Почему шифрование диска не решает всё

Encryption at rest — защита хранимых данных, например на диске или в резервной копии. **Encryption in transit** — защита передачи по сети. **Защита на уровне приложения** шифрует выбранные поля до записи в хранилище.

Шифрование диска помогает против определённого доступа к носителю, но приложение, законно читающее БД, обычно получает открытые значения. Поэтому оно не обязательно защищает от злоумышленника с правами SQL-чтения. А шифрование полей не защищает от процесса, который уже имеет право расшифровывать их. Механизм выбирают по модели угроз. [47]

> Проверка понимания: чем отличаются хеш, HMAC, цифровая подпись и шифрование? Для каждого назови, кто способен восстановить данные или создать допустимое проверочное значение.

III. БЕЗОПАСНОСТЬ

16. KMS, HSM и секреты: как сервер управляет ключами

Что такое KMS и как он работает

KMS, Key Management Service, — сервис управления криптографическими ключами. Он хранит или контролирует ключевой материал, проверяет право использования и выполняет разрешённые криптографические операции через API. **Ключевой материал** — собственно секретные байты ключа; **идентификатор ключа** — ссылка на управляемый объект, а не сам секрет.

В AWS KMS объект ключа включает материал и сведения для управления им. Приложение обращается к KMS от определённой машинной личности. KMS проверяет политику и выполняет действие; обычный симметричный KMS-ключ не скачивается приложением как файл. Это позволяет отделить право читать таблицу от права использовать ключ. [50]

Не стоит представлять KMS просто как «ещё одну таблицу с паролями». Его ценность — в контролируемом использовании ключа, управлении жизненным циклом и аудируемой границе доступа. Однако если атакующий захватил процесс с правом расшифрования, он может обращаться к KMS от имени этого процесса. Неэкспортируемость ключа не делает скомпрометированное приложение безопасным.

Что такое HSM

HSM, Hardware Security Module, — специализированный аппаратный модуль для защищённого хранения ключей и криптографических операций. Он создаёт аппаратную границу вокруг ключевого материала и применяет свои правила доступа. AWS CloudHSM — пример управляемой инфраструктуры таких модулей. [54]

KMS — уровень сервиса и политик управления; HSM — аппаратный механизм защиты и выполнения криптографии. KMS может использовать HSM внутри, но эти термины не являются синонимами. Для нашего учебного backend выбираем управляемый KMS; отдельную инфраструктуру HSM обсуждали бы при конкретном требовании к контролю ключей или криптографическим операциям.

KeyStore в серверной Java — программная абстракция хранения ключей и сертификатов. Файл KeyStore сам по себе не равен аппаратно защищённому модулю. Важно, где лежат ключевые байты, кто может их прочитать и какая реализация стоит за интерфейсом. Название класса не устанавливает границу безопасности. [76]

Envelope encryption: зачем шифровать ещё и ключ

Envelope encryption, конвертное шифрование, использует два уровня. **DEK, Data Encryption Key**, шифрует данные. **KEK, Key Encryption Key**, защищает сам DEK. В нашем варианте KMS-ключ играет роль верхнего ключа, а адрес заказа шифруется отдельным ключом данных. [77]

Почему так? Основной объём данных обрабатывает приложение или библиотека, а центральная система контролирует доступ к ключу расшифрования. AWS KMS `GenerateDataKey` возвращает открытый ключ данных для немедленного использования и его зашифрованную копию. KMS не хранит вместо приложения все такие data keys: зашифрованную копию сохраняет наша система. [51]

Сквозной пример для адреса:

```
1. Backend получает право использовать KMS-ключ.
2. Просит создать DEK для конкретного контекста.
3. Получает plaintext_DEK и encrypted_DEK.
4. Проверенная библиотека шифрует адрес с plaintext_DEK.
5. В БД сохраняются encrypted_DEK, шифротекст и параметры.
6. Открытый DEK больше не сохраняется и не журналируется.
```

При чтении backend загружает зашифрованный DEK, просит KMS расшифровать его, получает временный ключ данных, расшифровывает адрес и ограничивает время жизни открытых данных в памяти. Сам адрес при такой схеме не обязан отправляться в KMS. [51]

Управляемые языки не всегда позволяют гарантированно стереть все копии секрета из памяти. Поэтому не обещаем невозможного: ограничиваем копирование, срок жизни, дампы и логирование, используем библиотеку с подходящим поведением.

Что хранится рядом с зашифрованным полем

Наша учебная запись:

```
{
  "encryptionVersion": 1,
  "kmsKeyId": "<managed-key-id>",
  "encryptedDataKey": "<encoded-bytes>",
  "nonce": "<encoded-bytes>",
  "ciphertextWithTag": "<encoded-bytes>"
}
```

`encoded-bytes` означает текстовое представление байтов, а не ещё одно шифрование. Вместе с записью известны контекст заказа и выбранный алгоритм из доверенной версии формата. Алгоритм не выбирается без ограничений из пользовательского запроса.

AWS KMS поддерживает **encryption context** — набор значений, связываемых с криптографической операцией. Он не является секретным и может попадать в журналы, поэтому туда не помещают адрес, токен или иные закрытые данные. [52]

Контекст KMS и AAD локального шифрования — разные точки проверки. В нашем дизайне обе связываются с нужным объектом по явно описанным правилам. Чтобы расшифровать чужой адрес, недостаточно подменить `orderId` в HTTP-запросе: сначала проверяется доступ к заказу, затем формируется ожидаемый контекст из доверенных данных.

Ротация и потеря ключа

Ротация криптографического ключа — управляемая смена материала или ключа для будущего использования. Автоматическая ротация AWS KMS не означает, что все ранее записанные данные автоматически перешифрованы; прежний материал сохраняется для расшифрования соответствующих данных. [53]

Rewrap, перепаковка ключа данных, означает перешифрование DEK новым верхним ключом без обязательного изменения основного шифротекста. **Re-encryption данных** означает уже новое шифрование самих данных. Выбор зависит от причины ротации. Если скомпрометирован именно открытый DEK, одной перепаковкой того же DEK проблему не решить.

Перед удалением ключа выясняем, какие рабочие данные и резервные копии от него зависят: удаление необходимого ключа способно сделать данные невосстановимыми. [78] Скопировать зашифрованную БД без возможности восстановить ключи — не полноценное восстановление. Недоступность KMS также является эксплуатационным сценарием: наш сервис не переходит на запись открытых адресов ради обхода ошибки.

Secrets manager — другая задача

Secret, секрет, — значение, которое даёт возможность доступа или действия: API-ключ партнёра, пароль БД, ключ webhook. **Secrets manager** — сервис хранения, выдачи и смены таких значений по политике. В отличие от неэкспортируемого KMS-ключа, секрет часто требуется выдать приложению в открытом виде, чтобы оно предъявило его внешнему API. OWASP рассматривает централизованное хранение, жизненный цикл, аудит и автоматизацию управления секретами. [55]

Workload identity, машинная личность процесса, позволяет выдавать серверу права без постоянного общего секрета в исходном коде. **IAM, Identity and Access Management**, — система управления такими личностями и разрешениями. Наш worker получает только те права на секрет партнёра и KMS, которые нужны выпуску карты; сервис каталога дизайнов их не получает.

У Stripe различаются публикуемые и секретные ключи, тестовая и рабочая среда, а также ограниченные ключи. Серверный секрет не отправляется в мобильное приложение и не попадает в журнал запросов. [8]

Ротация секрета требует согласованного перехода: выдать новый, перевести потребителей, проверить работоспособность и отозвать старый. Простая смена значения без учёта работающих процессов может остановить интеграцию.

> Проверка понимания: где в твоей схеме находится открытый DEK, где верхний ключ, а где API-секрет партнёра? Почему к ним применяются разные способы защиты?

III. БЕЗОПАСНОСТЬ

17. Данные, сеть и опасные входы API

Начинаем с жизненного цикла данных

Классификация данных определяет, что является публичным, внутренним, персональным или секретным. **PII, Personally Identifiable Information**, — сведения, позволяющие идентифицировать человека в соответствующем контексте. Адрес доставки и связь заказа с пользователем требуют защиты, даже если в них нет номера карты.

Для каждого вида данных определяем цель, получателей, место хранения, срок и механизм удаления. **Retention policy**, политика хранения, — договорённость о том, сколько и зачем сохраняется информация. **Минимизация данных** означает не собирать и не копировать лишнее. Это часто сильнее, чем шифровать ненужную десятую копию. [47]

В нашем заказе адрес может встретиться в основной БД, outbox, inbox, HTTP-логах, отчётах поддержки и резервных копиях. Защита одной таблицы не контролирует остальные места.

Токенизация карточных данных

PAN, Primary Account Number, — основной номер платёжной карты. **Токенизация** заменяет чувствительный реквизит другим идентификатором, который применяется в ограниченном контексте. Это не обязательно шифрование и не гарантия бесполезности токена для злоумышленника.

В задаче выпуска карты провайдер возвращает уникальный token. До проектирования уточняем: это только ID карты, значение для операций над ней или секрет, позволяющий получить реквизиты? Нельзя считать его безопасным для логов только из-за слова token.

PCI DSS — стандарт защиты карточных данных и соответствующей среды. **Compliance**, соответствие требованиям, включает область применения, технические меры, процессы и подтверждение их выполнения; одна диаграмма его не доказывает. [63]

CVV/CVC — проверочный код карты. Для продавцов PCI SSC запрещает сохранять такие коды после авторизации, даже зашифрованными. Не переносим эту формулировку без разбора на все специальные процессы эмиссии: наш учебный сервис заказа не проектирует хранение CVV и должен уточнять применимые правила для фактической роли организации. [64]

TLS: как защищается соединение

TLS, Transport Layer Security, — протокол защищённого соединения. При установлении связи стороны согласуют параметры и получают ключи сеанса; сервер подтверждает свою личность сертификатом и владением соответствующим ключом. Дальнейший обмен защищается от чтения и незаметного изменения в рамках протокола. [59]

Сертификат связывает открытый ключ с именем или другой идентичностью и заверяется доверенной стороной. **CA, Certificate Authority**, — удостоверяющий центр. Клиент проверяет цепочку доверия и соответствие имени ожидаемому серверу. Отключение проверки сертификата не является допустимым способом «починить HTTPS». HTTPS — HTTP поверх TLS. [70]

mTLS, взаимный TLS, добавляет аутентификацию клиента сертификатом. Для связи сервисов это может подтвердить машинную личность, но не заменяет право именно этого сервиса выполнить именно эту бизнес-команду. Сервис с правильным сертификатом не становится автоматически администратором.

Reverse proxy, обратный прокси, принимает запрос перед приложением и пересылает дальше. **TLS termination** означает завершение защищённого соединения на этом промежуточном компоненте. Нужно отдельно решить, как защищён следующий участок и как передаётся проверенная личность. Заголовку `X-User-Id` от произвольного внешнего клиента доверять нельзя. RFC 9700 отдельно обсуждает границу доверия TLS-терминирующих прокси. [42]

SQL injection и mass assignment

SQL injection, внедрение SQL, возникает, когда пользовательские данные становятся частью исполняемой структуры запроса. Параметризованные запросы передают значения отдельно от SQL-кода. Выбор имени столбца или направления сортировки обычно проверяют по закрытому набору допустимых значений, а не пытаются безопасно склеить произвольный ввод. [57]

Mass assignment — непреднамеренная запись клиентских полей во внутреннюю модель через автоматическое связывание. Пользователь отправляет `status: "APPROVED"` или `feeMinor: 0`, а общий механизм переноса полей сохраняет это в объект. OWASP рекомендует явно ограничивать доступные для связывания поля. [71]

В нашем `CreateCardOrderRequest` принимаются дизайн и адрес. Поля владельца, провайдера, статуса и стоимости вычисляются либо проверяются сервером. Ограничения длины, размера тела, количества элементов и диапазонов нужны до дорогостоящей обработки. **Validation**, валидация, проверяет допустимость входа, но не заменяет авторизацию.

SSRF: сервер заставили обратиться не туда

SSRF, Server-Side Request Forgery, — ситуация, когда злоумышленник управляет адресом серверного обращения и добивается доступа к непредусмотренным ресурсам. Пример: пользователь передал URL картинки дизайна, а backend загрузил внутренний административный endpoint.

Защита включает ограничение допустимых адресов и протоколов, проверку перенаправлений, контроль разрешённых сетевых направлений и отказ от ненужной загрузки произвольных URL. OWASP отдельно разбирает адреса, DNS и опасность обхода ограничений. [58]

Allowlist — закрытый список разрешённых вариантов. **Egress control** — ограничение исходящих сетевых соединений. В учебном сервисе worker выпуска обращается к заранее настроенному провайдеру, а не к URL из клиентского заказа.

Логи и аудит

Лог — техническая запись о работе программы. Не следует журналировать Authorization-заголовки, пароли, секретные ключи или полные финансовые реквизиты. **Redaction**, удаление или маскирование чувствительных полей, выполняется до отправки в централизованное хранилище журналов. OWASP описывает исключаемые данные, защиту логов и сроки хранения. [56]

Для расследования обычно полезнее `operation_id`, номер попытки, безопасный код ошибки и длительность, чем полный request body. **Корреляционный идентификатор** связывает записи об одной операции между компонентами. У Stripe есть собственный Request ID, который можно сохранять для обращения в поддержку, не подменяя им наш ID бизнес-операции. [10]

Аудит значимого изменения сохраняем согласованно с ним или через надёжный механизм доставки. Обычный `print("transfer approved")` не гарантирует след аудита. Ограничиваем право изменять историю и проверяем действия привилегированных пользователей. Хеш-цепочка записей способна обнаруживать некоторые изменения, но без независимого доверенного закрепления не защищает от полного переписывания цепочки субъектом с соответствующим доступом.

> Проверка понимания: может ли секрет уйти через лог или outbox, хотя основное поле БД зашифровано? Пройди все копии, а не только основную таблицу.

IV. ПРАКТИКА И ЭКСПЛУАТАЦИЯ

18. Эксплуатация: скорость, восстановление и наблюдаемость

Кеш и реплика — не новый финансовый авторитет

Реплика — копия БД, получающая изменения от основного узла. **Primary**, основной узел, принимает записи в рассматриваемой схеме. **Replication lag**, задержка репликации, — отставание копии. Асинхронная реплика PostgreSQL может ещё не содержать недавно зафиксированные данные; синхронный режим меняет компромисс между ожиданием и сохранностью при отказах. [27]

В нашей архитектуре каталог дизайнов допускает кеш. Но разрешение нового списания проверяется в контролируемой актуальной модели остатка, а не на случайной реплике. Даже правильный `SELECT` не защищает инвариант, если читается неподходящая копия.

Read-your-writes, чтение собственных записей, — обещание, что после успешного изменения пользователь сможет прочитать как минимум это изменение. Если POST создал заказ, а следующий GET попал на отставшую реплику и вернул 404, клиент может ошибочно начать новый заказ. Для критических переходов наш API читает primary или использует иной явно определённый механизм ожидания нужной версии.

Eventual consistency, согласованность с задержкой, означает, что копии сходятся при выполнении условий системы, но не обязательно совпадают немедленно. Это допустимый выбор для отображения истории, когда свежесть обозначена, но не оправдание проверки доступного остатка на заведомо устаревшей копии.

Масштабируем именно ограниченное место

Горизонтальное масштабирование добавляет экземпляры сервиса. **Load balancer**, балансировщик нагрузки, распределяет обращения между ними. **Stateless handler**, обработчик без локально обязательного состояния, хранит важный контекст вне памяти одного экземпляра, чтобы другой мог продолжить запрос или операцию.

В нашем проекте HTTP-обработчики можно увеличивать отдельно от workers. Но дополнительные workers не помогут, если провайдер разрешает только ограниченное число запросов. **Backpressure**, обратное давление, — ограничение приёма или скорости работы, чтобы следующий этап не был перегружен. В нашем варианте это контролируемая очередь, лимит внешних вызовов и честный статус ожидания.

Connection pool, пул соединений, — ограниченный набор переиспользуемых подключений к БД. Десять новых экземпляров с сотней соединений каждый могут перегрузить БД. Поэтому масштабирование API сопровождается общим бюджетом подключений, ограничением конкуренции и измерением ожидания блокировок.

Sharding, разделение данных по независимым узлам, помогает распределять объём или нагрузку, но усложняет операции между частями. Для нашего запуска он не нужен. Если будущий перевод затрагивает счета разных частей, потребуется новое обоснование границы финансовой транзакции, а не простая замена адреса БД.

Метрики, логи и трассировка

Метрика — числовое измерение во времени: частота ошибок, длительность, размер очереди. **Trace**, трасса, связывает шаги одного взаимодействия; **span** — отдельный измеренный участок, например запрос к провайдеру. OpenTelemetry описывает такие сигналы и их совместное применение для наблюдения за системой. [60]

SLI, Service Level Indicator, — измеряемый показатель качества. **SLO, Service Level Objective**, — целевое значение этого показателя за определённый период. **Error budget**, бюджет ошибок, — допустимая доля нарушения выбранной цели. Это разные понятия; «у нас есть мониторинг» не задаёт ни одного из них. [61]

Для учебного продукта разделим цели принятия заявки и завершения выпуска. Быстрый `202` не доказывает, что карта будет выпущена вовремя. Измеряем возраст старейшей необработанной outbox-записи, число операций с неизвестным исходом, задержку webhook, долю конфликтов назначения и размер необъяснённых финансовых расхождений.

p95, 95-й процентиль длительности, — значение, не превышенное примерно 95% измеренных запросов в выбранной выборке. Средняя задержка может скрывать редкие, но болезненные ожидания. Однако и p95 принятия команды не заменяет измерения полного времени бизнес-операции.

Alert, оповещение, сообщает о ситуации, требующей действия. Для него нужен **runbook** — понятная инструкция дежурному: какие данные проверить, какие действия безопасны и чего нельзя делать. Оповещение «слишком много UNKNOWN» без способа поиска операций мало помогает.

Backup, RPO и RTO

Backup, резервная копия, сохраняется для восстановления после потери или повреждения данных. Реплика не заменяет её: ошибочное удаление может повториться на репликах.

RPO, Recovery Point Objective, — максимально допустимая потеря данных, выраженная через время. **RTO, Recovery Time Objective**, — целевой срок восстановления работоспособности. Это требования, которые затем проверяют выбранной схемой копирования, хранения ключей и практическими учениями.

PITR, Point-in-Time Recovery, — восстановление БД на определённый момент. PostgreSQL использует базовую резервную копию и последовательность WAL-записей. Нужно сохранять обе части и проверять процедуру, а не только наличие файла с датой. [26]

Рассмотрим неприятный сценарий. Восстановились на 10:00, но провайдер успел выполнить заказы с 10:00 до 10:05. Локальная БД о них больше не знает. Если сразу включить всех workers и принять повторы как новые действия, можно создать дубли. Поэтому в нашем плане восстановления есть пауза внешних изменений, восстановление связей с партнёром и сверка, а не только запуск PostgreSQL.

Тест восстановления включает зашифрованные данные, KMS-права, секреты, записи идемпотентности, outbox, inbox и финансовую историю. «БД поднялась» — лишь промежуточный результат.

Изменение схемы без несовместимого перехода

Миграция — управляемое изменение схемы или представления данных. **Expand–contract** — подход, при котором сначала добавляют совместимый новый формат, переводят чтение и запись, а затем удаляют старый.

Пример: добавили `recoveryStatus`, но старый worker продолжает считать любую ошибку отказом. Одной миграции столбца мало; нужны совместимость версий и запрет старого пути исполнения после перехода. То же относится к изменению нормализации idempotency fingerprint: старые повторы должны проверяться по корректной версии правил.

Версия API фиксирует определённый контракт, а не только номер в URL. У Stripe изменение версии SDK и API требует учитывать документированную политику совместимости. Для интеграции фиксируем ожидаемую версию и проверяем обновление на сохранённых тестовых примерах. [11]

> Проверка понимания: после восстановления БД из копии внешняя система тоже вернулась в прошлое? Если нет, где в плане восстановления находится сверка уже совершённых действий?

IV. ПРАКТИКА И ЭКСПЛУАТАЦИЯ

19. Два полных решения: карта и ATM

Заказ карты: границы учебной задачи

Спроектируем выпуск физической карты через одного партнёра. Пользователь выбирает дизайн, подтверждает адрес, отправляет заказ и видит его состояние. Выпуск может продолжаться после ответа API. Партнёр поддерживает стабильный ключ операции и поиск результата; это явное учебное допущение, которое на интервью требуется проверить.

Стоимость выпуска и учёт денег рассматриваются как дополнительный модуль, только если они входят в продуктовый scope. Не нужно строить собственный ledger для бесплатного заказа лишь потому, что продукт банковский.

Начальная архитектура — backend с модулями заказов, доступа и интеграции; PostgreSQL; фоновые workers; защищённое хранение секретов и ключей. Внешние уведомления о статусе ускоряют обновление, но работоспособность не зависит от единственного webhook.

Какие данные и почему храним

`card_orders` содержит владельца, область, состояние выпуска, снимок параметров и связь с провайдерским объектом. `api_operations` связывает повторяемую клиентскую команду с заказом. `provider_operations` хранит стабильную идентичность внешнего действия и процесс выяснения результата. `provider_attempts` хранит технические попытки.

`outbox` хранит надёжно принятые задания. `inbox` — принятые внешние события. `audit_events` — значимые действия и основания их выполнения. Это разные роли данных. В небольшом проекте их не нужно превращать в семь микросервисов.

Адрес шифруется с подходящей привязкой к заказу. Секреты партнёра не записываются в перечисленные таблицы. Неизменяемые финансовые значения, если они есть, отделены от редактируемого описательного текста.

API сценария

```
GET  /v1/card-designs
GET  /v1/delivery-addresses
POST /v1/delivery-addresses
POST /v1/card-orders
GET  /v1/card-orders/{orderId}
GET  /v1/card-orders?status=active
POST /v1/card-orders/{orderId}/cancellation-requests
```

Список активных заказов помогает восстановиться, если пользователь утратил локальное состояние и не знает ID. Все чтения ограничены текущими правами. **Cursor pagination**, курсорная навигация, передаёт позицию следующей порции результатов, а не предполагает неизменность номера страницы в меняющемся списке. Для нашего списка можно использовать устойчивый порядок по времени и ID; курсор не даёт права читать чужие записи.

Отмена моделируется запросом, потому что партнёр мог уже начать необратимую работу. Успешное принятие `cancellation-request` не означает гарантированную отмену выпуска. Такой контракт честнее, чем моментальный `status = CANCELED` без ответа партнёра.

Путь одной команды

Сервер проверяет личность и доступ к выбранному адресу. В короткой транзакции создаёт область ключа, заказ со снимком параметров, запись внешней операции и outbox-задание. После фиксации возвращает `202` и ID.

Worker забирает задание, проверяет, что действие ещё допустимо, и вызывает партнёра с сохранённым ключом. При известном успехе сохраняет связь с картой и переводит выпуск в `ISSUED`. Доставку ведёт отдельно. При сетевой неопределённости назначает проверку исхода, а не повтор с новым ключом.

Webhook после проверки попадает в inbox. Его обработчик и ответ первоначального запроса могут прийти одновременно, поэтому оба применяют один контролируемый алгоритм перехода. Внешний ID карты должен иметь правильную уникальность внутри аккаунта провайдера. Повтор не создаёт вторую карту и второе финансовое событие.

Периодический процесс ищет слишком долго незавершённые операции. Для каждого результата есть авторитетное подтверждение или открытый случай сверки. Поддержка видит ID и состояние, но не получает неограниченный доступ к секретам.

Как обрабатываются пять отказов

Если потерян клиентский ответ, повтор ключа возвращает прежний заказ. Если worker упал до обращения, задание подхватывается снова. Если worker упал после успешного внешнего действия, новый обработчик выясняет прежний результат по той же идентичности. Если webhook повторился, inbox и бизнес-ограничения исключают повторный эффект. Если KMS недоступен, операция откладывается либо возвращает безопасную ошибку; адрес не начинает храниться открыто.

Это не «все ошибки поймаем в catch». Для каждого разрыва существует сохранённое состояние и определённый следующий шаг.

АТМ: задача шире назначения

Исходная бизнес-цель — перестать ездить к каждому АТМ без знания потребности в пополнении. **АТМ** — банкомат. **Телеметрия** — данные о состоянии устройства, поступающие от него или обслуживающего провайдера. Начальная модель должна отвечать, как телеметрия превращается в задачу с нужной суммой.

Предположим, провайдер передаёт остаток и время измерения. Наш backend сохраняет `observed_at`, то есть время наблюдения, и `received_at`, время получения. Поздно пришедшее старое измерение не должно автоматически заменить новое. Конкретное сравнение зависит от гарантируемой версии или временной модели источника.

Freshness, свежесть данных, — насколько наблюдение подходит для текущего решения. При слишком старом остатке наша система показывает проблему свежести и не притворяется, что вычисленная потребность актуальна. Порог и стратегия получения данных согласуются с продуктом.

Создание и жизненный цикл задачи

Учебный модуль потребности рассчитывает сумму пополнения по явно согласованному правилу. Мы не выдаём простую формулу порога за полноценную оптимизацию кассет, номиналов и маршрутов. **Цикл пополнения** — конкретный эпизод потребности; его ID позволяет отличать новую задачу через неделю от повторных данных о сегодняшней.

Можно принять правило «одна активная задача на АТМ», но это допущение нужно подтвердить: некоторые процессы могут допускать несколько разных работ. При принятом правиле используем соответствующее ограничение уникальности и обновляем существующую задачу, а не создаём новую на каждое измерение.

```
GET  /v1/atms
GET  /v1/atms/{atmId}
GET  /v1/refill-tasks
GET  /v1/refill-tasks/{taskId}
POST /v1/refill-tasks/{taskId}/assignment
POST /v1/refill-tasks/{taskId}/completion
```

Назначение использует условное обновление из главы 5. Роль, организация и допустимость конкретной задачи проверяются сервером. Отдельный idempotency key для простого назначения не объявляем обязательным, если повтор уже корректно распознаётся по состоянию и побочные эффекты защищены.

Завершение принимает только уполномоченный исполнитель и только в допустимом состоянии. Если сотрудник указал фактически внесённую сумму, она не становится автоматически подтверждённым остатком АТМ. Нужно решить, требуется ли сверка с устройством, внешней системой или другим доказательством.

Offline и alerts как требования к серверу

Offline-режим означает работу клиента без связи. Backend не может мгновенно подтвердить эксклюзивное назначение двум отключённым друг от друга клиентам. В нашем варианте сохранённая попытка означает «ожидает подтверждения», а не гарантированную собственность задачи. После восстановления связи сервер разрешает конфликт.

Создание задачи сопровождается надёжной записью события для уведомления. Если механизм доставки alerts находится вне scope, фиксируем эту границу: событие, получатели и доступ к актуальным данным всё равно должны быть определены, но инфраструктуру доставки подробно не проектируем.

Для пяти АТМ не нужен отдельный сервис на каждый прямоугольник. Сначала проверяем свежесть данных, сумму, исключительность назначения и восстановление после сбоев. Именно эти свойства устраняют исходную бизнес-проблему.

> Проверка понимания: пройди обе архитектуры без названий технологий. Объясни, откуда приходит факт, кто имеет право изменить его и как восстанавливается неизвестный результат.

IV. ПРАКТИКА И ЭКСПЛУАТАЦИЯ

20. Практикум: проверить механизм, а не запомнить название

Как работать с заданиями

Интеграционный тест проверяет совместную работу компонентов, например реального PostgreSQL и нашего обработчика. **Fault injection**, внесение сбоев, намеренно разрывает процесс в выбранной точке. **Property-based testing**, проверка свойств на множестве сгенерированных примеров, проверяет инвариант, а не только один заранее подготовленный результат.

Ниже вопросы сначала стоит решить самостоятельно. Предлагаемые ответы относятся к учебной модели доклада. Для интеграции с провайдером дополнительно используем его тестовую среду; Stripe предоставляет тестовые значения и сценарии. Они не заменяют испытание наших собственных падений и гонок. [12]

1. Сто одинаковых запросов

Условие. Сто конкурентных запросов создания карты имеют одинаковые scope, ключ и параметры. Некоторые получают обрыв соединения.

Ожидаемое свойство. В БД один принятый заказ для этой команды, одна связь с внешним выпуском, а каждый успешный ответ указывает на тот же заказ. Различаться могут технические результаты ожидания, но не бизнес-идентичность.

Что проверять. Уникальное ограничение, согласованную фиксацию заказа и ключа, сохранение outbox и восстановление после потери ответа. Такой тест нужно проводить между несколькими процессами, иначе локальная блокировка может скрыть ошибку распределённого исполнения.

2. Один ключ, другой адрес

Условие. Второй запрос повторяет ключ, но изменяет адрес.

Ответ. Это не повтор той же команды. Сервер возвращает конфликт параметров и не меняет созданный заказ. Новое пользовательское намерение проходит отдельную процедуру. При этом изменение порядка полей JSON не должно ложно считаться изменением адреса.

3. Два сотрудника и одна задача

Условие. Два разных пользователя одновременно назначают себе задачу.

Ответ. Только один переход из AVAILABLE в ASSIGNED фиксируется. Другой получает актуальный конфликт в пределах своих прав. Проверяем состояние строки, историю и число уведомлений, а не только HTTP-коды.

4. Сервер БД ответил неразборчиво после COMMIT

Условие. Приложение потеряло связь сразу после отправки COMMIT и не знает его исхода.

Ответ. После восстановления соединения ищем операцию по устойчивой идентичности. Не считаем отсутствие подтверждения доказательством отката. Повтор той же команды должен быть безопасен независимо от того, состоялась фиксация или нет.

5. Карта выпущена, worker погиб

Условие. Партнёр создал карту, а локальная запись успеха не сохранилась.

Ответ. Продолжаем прежнюю provider operation. Получаем существующий результат или выполняем допустимый повтор с тем же ключом в пределах контракта. Новый случайный ключ на каждой попытке провалит тест.

6. Событие пришло дважды и не по порядку

Условие. Сначала доставлен финальный статус, затем старый промежуточный; оба сообщения продублированы.

Ответ. Конечный объект не откатывается назад. Транспортные дубли не создают повторный эффект. Подпись проверяется на каждой новой доставке, а состояние изменяется по контролируемой модели, а не порядку прибытия.

7. Два перевода по 80 при остатке 100

Условие. У переводов разные ключи и корректные подтверждения.

Ответ. При запрете перерасхода нельзя успешно зарезервировать 160. Проверяем финансовую конкурентность отдельно от дедупликации. Property-based тест дополнительно может генерировать последовательности резервов, исполнений и освобождений и проверять сохранение инварианта.

8. Пользователь А знает ID заказа Б

Условие. Токен А корректен, а в пути запроса указан чужой заказ.

Ответ. Доступ запрещён по правилам объекта. Такой же тест выполняется для списка, экспорта, повторного ответа идемпотентности и служебного endpoint статуса. OWASP предлагает автоматизировать матрицу «роль — действие — данные», чтобы изменения функциональности не ломали авторизацию незаметно. [66]

9. Сменили получателя после подтверждения

Условие. Сохранённое разрешение относится к версии 7, worker пытается исполнить версию 8.

Ответ. Исполнение не разрешается этим доказательством. Проверяется неизменяемая связь подтверждения с параметрами. Дополнительное окно биометрии не исправляет неверно спроектированную связь на сервере.

10. Потерян доступ к KMS

Условие. Сервис имеет зашифрованный адрес, но не может получить DEK.

Ответ. Он не пишет открытый адрес в обход защиты и не подставляет неверные данные. Операция получает восстанавливаемое техническое состояние. Проверяем также отсутствие ключей и plaintext в журналах ошибки.

11. Восстановление из вчерашней копии

Условие. Внешний партнёр выполнил часть операций после момента копии.

Ответ. Сначала устанавливаем, какие внешние эффекты уже были, и восстанавливаем связи. Не запускаем повторные изменения без проверки. Успешный старт БД ещё не означает корректного восстановления продукта.

12. Доклад на собеседовании за пять минут

Попробуй объяснить одну операцию в таком порядке: пользовательское намерение; инвариант; проверка доступа; граница транзакции; конкурентный конфликт; внешний вызов; неизвестный результат; способ восстановления; безопасный ответ клиенту.

Хорошее объяснение не обязано содержать все термины этого документа. Оно должно содержать те механизмы, которые требуются именно задаче. Для назначения одной ATM-задачи условное обновление может быть важнее многоступенчатой saga. Для внешнего платежа одного UPDATE уже недостаточно.

> Итоговый вопрос: если убрать один компонент из твоей схемы, какое проверяемое свойство перестанет выполняться? Если ответа нет, компонент либо лишний, либо недостаточно понятен.

СПРАВОЧНЫЙ РАЗДЕЛ

Приложение A. Карта понятий для повторения

Это не новые темы, а короткие напоминания после основного текста. Полные объяснения находятся в указанных главах.

Понятие	Что нужно помнить	Глава
Бизнес-операция и запрос	Одно намерение может иметь несколько сетевых попыток	1
Инвариант	Проверяемое правило для всех допустимых состояний	1
Money и minor units	Сумма требует валюты, масштаба, диапазона и округления	2
Машина состояний	У каждого перехода есть условие и подтверждение	3
ACID и COMMIT	Общая фиксация ограничена участниками транзакции	4
Условное обновление	Проверка и изменение не разделены гонкой	5
Idempotency key	Идентичность повтора, а не проверка доступа	6
Timeout	Прекращение ожидания, не доказательство отказа	7
Outbox	Надёжно сохраняет необходимость исходящего действия	8
Inbox	Надёжно принимает событие и помогает исключать повторы	8–9
Saga и компенсация	Несколько локальных шагов вместо общего отката	8
Webhook	Внешний запрос требует проверки и надёжного приёма	9
Ledger и резерв	Учёт денег и запрет повторного расходования — разные задачи	10
Reconciliation	Установление соответствия независимых записей	11
Authentication / authorization	Личность и право на объект проверяются отдельно	12
OAuth / OIDC / JWT	Протокол доступа, протокол входа и формат токена	13
Transaction authorization	Подтверждение связано с конкретными параметрами	14
Encryption / hash / signature	Скрыть, вычислить отпечаток, проверить происхождение	15
KMS / HSM / secrets manager	Управление ключами, аппаратная защита и выдача секретов	16
DEK / KEK / AAD	Ключ данных, защита ключа и связанный контекст	15–16
TLS / mTLS	Защита канала не заменяет бизнес-авторизацию	17
Audit / logs	След значимых действий отличается от диагностики	17
SLO / RPO / RTO	Цель качества, допустимая потеря и срок восстановления	18

СПРАВОЧНЫЙ РАЗДЕЛ

Приложение Б. Какие стандарты относятся к backend

OWASP — открытое сообщество, выпускающее материалы по безопасности приложений. **ASVS, Application Security Verification Standard**, — набор проверяемых требований к безопасности приложений. Для backend он помогает превратить слова «безопасное API» в конкретные требования и проверки. Это более прямой ориентир для серверной части, чем MASVS — стандарт проверки безопасности мобильных приложений. [62]

FAPI 2.0 Security Profile — профиль повышенной защищённости OAuth для соответствующих API-сценариев. Он задаёт требования к серверам авторизации, серверам ресурсов и confidential clients — клиентам, способным защищать свои учётные данные. Профиль не заменяет финансовый учёт и не распространяется автоматически на любой мобильный клиент. [65]

PCI DSS относится к своей области карточных данных и среды. OAuth относится к доступу. ASVS — к проверке безопасности приложения. Ни один из них не доказывает правильность всех бизнес-проводок, а balanced ledger не доказывает безопасность ключей. Каждый инструмент отвечает на свой класс вопросов.

Внедрение конкретных нормативных требований зависит от продукта, роли организации, рынка и договоров. Этот доклад объясняет технические механизмы; он не присваивает проекту статус соответствия и не заменяет разбор области применения с профильными специалистами.

Последняя мысль

Надёжный backend не обязан знать результат любой операции немедленно. Он обязан различать известное и неизвестное, не нарушать инварианты во время ожидания, сохранять возможность восстановления и не выдавать клиенту уверенность, которой у системы пока нет.

Для финтеха особенно важно, что безопасность и корректность не живут в отдельном прямоугольнике. Они проявляются в типе суммы, области ключа, SQL-условии, привязке подтверждения, журнале попыток, доступе к KMS и инструкции по восстановлению. Именно эту связь нужно уметь объяснять.

ПЕРВОИСТОЧНИКИ

Источники

Официальные первоисточники, проверенные при подготовке 2 октября 2026 года. Для SQL выбран раздел документации PostgreSQL 18, для числовых примеров Java — Java SE 21; это не заявление о необходимости использовать именно эти версии. Ссылки документируют механизмы и продуктовые контракты, а не подтверждают внутреннее устройство Revolut.

01 Stripe — Idempotent requests, API v1
docs.stripe.com

02 Stripe — API v2 overview: Idempotency
docs.stripe.com

03 Stripe — Advanced error handling
docs.stripe.com

04 Stripe — The Payment Intents API
docs.stripe.com

05 Stripe — PaymentIntent lifecycle
docs.stripe.com

06 Stripe — Webhooks
docs.stripe.com

07 Stripe — Supported currencies
docs.stripe.com

08 Stripe — API keys
docs.stripe.com

09 Stripe — Rate limits
docs.stripe.com

10 Stripe — Request IDs
docs.stripe.com

11 Stripe — API versioning
docs.stripe.com

12 Stripe — Testing
docs.stripe.com

13 IETF — RFC 9110: HTTP Semantics
www.rfc-editor.org

14 IETF — RFC 9457: Problem Details for HTTP APIs
www.rfc-editor.org

15 Oracle — BigDecimal, Java SE 21
docs.oracle.com

16 PostgreSQL 18 — Numeric Types
www.postgresql.org

17 PostgreSQL 18 — Transactions
www.postgresql.org

18 PostgreSQL 18 — Transaction Isolation
www.postgresql.org

19 PostgreSQL 18 — Explicit Locking
www.postgresql.org

20 PostgreSQL 18 — Constraints
www.postgresql.org

21 PostgreSQL 18 — INSERT, ON CONFLICT
www.postgresql.org

22 PostgreSQL 18 — SELECT, SKIP LOCKED
www.postgresql.org

23 PostgreSQL 18 — Partial Indexes
www.postgresql.org

24 PostgreSQL 18 — Row Security Policies
www.postgresql.org

25 PostgreSQL 18 — Write-Ahead Logging
www.postgresql.org

26 PostgreSQL 18 — Continuous Archiving and Point-in-Time Recovery
www.postgresql.org

27 PostgreSQL 18 — Log-Shipping Standby Servers
www.postgresql.org

28 PostgreSQL 18 — Non-Durable Settings
www.postgresql.org

29 AWS — Transactional outbox pattern
docs.aws.amazon.com

30 RabbitMQ — Consumer Acknowledgements and Publisher Confirms
www.rabbitmq.com

31 Apache Kafka 4.1 — Design: message delivery semantics
kafka.apache.org

32 AWS — Control and limit retry calls
docs.aws.amazon.com

33 AWS — Circuit breaker pattern
docs.aws.amazon.com

34 AWS — Saga orchestration pattern
docs.aws.amazon.com

35 Modern Treasury — Ledger Transactions Overview
docs.moderntreasury.com

36 Modern Treasury — Debits and Credits
docs.moderntreasury.com

37 Modern Treasury — Transaction Status and Balances
docs.moderntreasury.com

38 Modern Treasury — Account Reconciliation
docs.moderntreasury.com

39 OWASP — Authorization Cheat Sheet
cheatsheetseries.owasp.org

40 OWASP — API1:2023 Broken Object Level Authorization
api-security.owasp.org

41 IETF — RFC 6749: OAuth 2.0 Authorization Framework
www.rfc-editor.org

42 IETF — RFC 9700: Best Current Practice for OAuth 2.0 Security
www.rfc-editor.org

43 IETF — RFC 8725: JSON Web Token Best Current Practices
www.rfc-editor.org

44 IETF — RFC 7662: OAuth 2.0 Token Introspection
www.rfc-editor.org

45 OpenID Foundation — OpenID Connect Core 1.0
openid.net

46 OWASP — Transaction Authorization Cheat Sheet
cheatsheetseries.owasp.org

47 OWASP — Cryptographic Storage Cheat Sheet
cheatsheetseries.owasp.org

48 OWASP — Key Management Cheat Sheet
cheatsheetseries.owasp.org

49 OWASP — Password Storage Cheat Sheet
cheatsheetseries.owasp.org

50 AWS KMS — KMS keys
docs.aws.amazon.com

51 AWS KMS — Generate data keys
docs.aws.amazon.com

52 AWS KMS — Encryption context
docs.aws.amazon.com

53 AWS KMS — Rotate keys
docs.aws.amazon.com

54 AWS CloudHSM — What is AWS CloudHSM?
docs.aws.amazon.com

55 OWASP — Secrets Management Cheat Sheet
cheatsheetseries.owasp.org

56 OWASP — Logging Cheat Sheet
cheatsheetseries.owasp.org

57 OWASP — SQL Injection Prevention Cheat Sheet
cheatsheetseries.owasp.org

58 OWASP — Server-Side Request Forgery Prevention Cheat Sheet
cheatsheetseries.owasp.org

59 IETF — RFC 8446: TLS 1.3
www.rfc-editor.org

60 OpenTelemetry — Signals
opentelemetry.io

61 Google SRE — Service Level Objectives
sre.google

62 OWASP — Application Security Verification Standard (ASVS)
owasp.org

63 PCI SSC — PCI Data Security Standard
www.pcisecuritystandards.org

64 PCI SSC — Are merchants allowed to request card-verification codes/values?
www.pcisecuritystandards.org

65 OpenID Foundation — FAPI 2.0 Security Profile
openid.net

66 OWASP — Authorization Testing Automation Cheat Sheet
cheatsheetseries.owasp.org

67 Stripe — Refund and cancel payments
docs.stripe.com

68 Stripe — How disputes work
docs.stripe.com

69 Stripe — Place a hold on a payment method
docs.stripe.com

70 OWASP — Transport Layer Security Cheat Sheet
cheatsheetseries.owasp.org

71 OWASP — Mass Assignment Cheat Sheet
cheatsheetseries.owasp.org

72 OWASP — Session Management Cheat Sheet
cheatsheetseries.owasp.org

73 IETF — RFC 5116: Authenticated Encryption
www.rfc-editor.org

74 IETF — RFC 2104: HMAC
www.rfc-editor.org

75 IETF — RFC 7519: JSON Web Token
www.rfc-editor.org

76 Oracle — KeyStore, Java SE 21
docs.oracle.com

77 AWS Encryption SDK — Concepts
docs.aws.amazon.com

78 AWS KMS — Delete a key
docs.aws.amazon.com